[University of Bedfordshire]

SOFTWARE DEVELOPMENT PROJECT REPORT

ELEVATOR CONTROL SYSTEM

Twin Lift Control Room: a two-elevator simulator with a safety interlock and MySQL event log

C# · .NET 10 · Windows Forms · MySQL

Submitted by

Saurav Bogati

Student ID: [2533511]

Programme: Desktop Applications Development and Software Engineering Supervisor / Module Leader **Manish sir**

Submission date: 11/2/2026

Declaration

I declare that this project report, titled “Elevator Control System”, is my own work. The system described was designed and implemented by me as part of the software development project for Desktop application management system. Where I have used documentation, libraries or ideas from other sources, they are acknowledged in the References section. This report has not been submitted, in whole or in part, for any other award or module.

Name: Saurav Bogati

Date: 11/2/2026

Acknowledgement

Thanking my supervisor, Mr. Manish sir , for his assistance and valuable suggestions throughout the course of this project and the faculty members for providing lectures on object-oriented programming, database management, and software engineering.

Thanking my classmates and my family for discussing various issues with them and being patient with me, and last but not least, thanking the authors of the open source documentation on C#,.NET, windows form application, and MySQL that helped me to learn about these technologies through this project.

Abstract

This project report highlights the implementation of an Elevator Control System (ECS), a desktop application referred to as "Twin Lift Control Room". This ECS is designed, developed and tested using C#.NET 10 and is a simulation application of 2 independent lifts: Lift A and Lift B that are responsible for transporting passengers on the Ground Floor (G) and the First Floor (1). This project makes use of Microsoft Windows Forms for designing and development while all events in this system are logged into a MySQL database using the MySqlConnector.

Each elevator is simulated by an individual object consisting of its position, request queue, 20 milliseconds timer, cabin door and one landings door at each floor level. The lift can be moved only if each door is closed as well as the loading does not exceed 800 kilograms – these conditions are checked on each movement tick. Emergency Stop, Alarm, Reset and Overload Protection are supported in this application. The ECS employs State Pattern (States are: Idle, Moving, and Emergency), as well as Observer Pattern so the dashboard reacts to any event as well as database logging mechanism without being dependent on each other. All database transactions run on the BackgroundWorker.

Contents

Declaration i
Acknowledgement ii
Abstract iii
1. Introduction 1
2. Problem Statement 1
2.1 Major Challenges 1
3. Proposed Solution 1
4. Aims and Objectives 2
4.1 Aim 2
4.2 Objectives 3
5. Project Planning and Requirements 3
5.1 Weekly Schedule 3
5.2 Functional Requirements 3
5.3 Non-Functional Requirements 4
6. System / Technical Requirements 5
6.1 Hardware Requirements 5
6.2 Software Requirements 5
7. System Design 7
7.1 Use Case Diagram 7
7.2 Class Diagram 7
7.3 ER Diagram 9
7.4 Activity Diagram 10
8. Tech Stack Implementation 11
8.1 Language Used 11
8.2 Object-Oriented Programming Implementation 12
8.3 Design Patterns Implemented 13
8.3.2Observer pattern 14
8.3.3Producer–consumer processing (not a formal GoF pattern) 14
8.3.4Separation of responsibilities 14
8.4 Database Used 14
9. Prototype 15
9.1 Main Window 16
9.2 Lift Control Card 16
9.3 Shaft, Cabin and Doors 19
9.4 Elevator Movement and Door Behaviour 19
9.5 Safety Behaviour 20
9.6 Event Log 21
10. Testing and Evaluation 22

10.1 Black Box Testing 22
10.2 White Box Testing 30
Summary 38
11. Critical Thinking 38
11.1 Advantages 38
11.2 Limitations 39
11.3 Future Improvements 39
12. Conclusion 39
14. Marking Matrix with Self-Assessment 40
15. Appendices 41
Appendix A – GitHub Link to Complete Source Code 41
.Appendix B – Use Case Diagram 42
Appendix C – Additional Screenshots 42
Appendix D – Additional Supporting Information 44

Table of Figures

Figure 1. Layered architecture of the Twin Lift Control Room 2
Figure 2. Use case diagram of the Twin Lift Control Room 7
Figure 3. Class diagram (1 of 2): control logic and design patterns 8
Figure 4. Class diagram (2 of 2): user interface and data access 9
Figure 5. ER diagram of the elevator_log table and its view 9
Figure 6. Activity diagram of a floor request in one lift 11
Figure 7. State machine of one lift (State pattern) 14
Figure 8. Main window of the Twin Lift Control Room with both lifts idle 16
Figure 9. Control card of Lift A while idle..... 17
Figure 10. Lift shaft with the cabin at the first floor and both doors open 17
Figure 11. Lift A in emergency stop with Lift B still working..... 18
Figure 12. Overload message on a lift control card 18
Figure 13. Event log filtered for Lift A 18
Figure 14. Use case diagram (repeated from Section 7.1) 25
Figure 15. Additional screenshot (for example Lift B travelling down while Lift A is idle) 26

List of Tables

Table 1. Major challenges and where the project addresses them 1
Table 2. Suggested weekly project schedule (edit to match the real timeline) 3
Table 3. Functional requirements 4
Table 4. Non-functional requirements 4
Table 5. Recommended hardware (suggested values, not benchmarked) 5
Table 6. Software requirements 5
Table 7. Technologies used 12
Table 8. Object-oriented principles and their implementation 12
Table 9. The three state classes 13
Table 10. Layers and what they know 14
Table 11. Table elevator_log 15
Table 12. Components of one lift control card 16
Table 13. Timing and sequence of one trip (values from the source cde) 17
Table 14. Black box test cases 19
Table 15. White box test cases 20
Table 16. Self-assessment against suggested criteria (suggested values) 23
Table 17. Use case descriptions 25
Table 18. Files of the Elevator_Twin project 26

1. Introduction

The Twin Lift Control Room Simulator is an application software that simulates a two-lift system and presents all the elevators along with their movements, doors, passenger call information, etc., in one single window. It keeps a record of all the events through MySQL. The software project uses major software development concepts such as OOP, design patterns, database management, concurrency, etc.

2. Problem Statement

A multi-elevator building must have software capable of handling requests for floor changes, transporting each elevator car safely, opening and closing doors appropriately, responding to emergencies, and maintaining logs of activity. Without proper structure, this kind of software ends up as a web of code, with a frozen display while storing data, and hazardous situations like an open door of an elevator car that is moving.

2.1 Major Challenges

Table 1. Major challenges and where the project addresses them

Challenge	Why it is difficult	Addressed in
Independent operation	Two lifts must run at the same time without one operating or disturbing the other.	One `Elevator`, timer, controller and `LiftUnit` per lift (Section 7.2)
Safety	A cabin must never move with a door open, and an emergency stop must freeze everything at once.	Door interlock and `EmergencyState` (Section 8.2, 8.3)
Smooth animation	Movement and door opening are animated every 20 ms, so nothing slow may run on the interface thread.	Timer-driven model; database work on a `BackgroundWorker` (Section 8.3)
Request handling	Requests can arrive at any moment, including while the cabin is moving.	Request queue and dispatcher in `Elevator` (Section 7.4)
Traceability	Every action should be stored and be searchable later.	`LogObserver`, `LogRepository`, table `elevator_log` (Section 8.4)
Maintainability	Interface, control logic and data access must stay separate so each can change alone.	Layered structure and design patterns (Sections 3 and 8)

3. Proposed Solution

The suggested solution includes a dashboard created using Windows Forms with a backing objectoriented model of the simulation. The Elevator class takes care of position, doors, request queue and timer of one cabin only. Elevator behavior when the timer expires is controlled by an object of a specific state (IdleState, MovingState or Emergency State). Whatever action happens, it calls the Notify function that reports about the event to all the observers: the Lift Unit control is redrawn, and an instance of a Log Observer passes the event to the Log Repository, which saves it to MySQL database in a separate thread. The dashboard window includes two Lift Unit controls. Each control creates its own Elevator and Elevator Controller objects; that is why any action on Lift A cannot affect Lift B. See Figure 1 for the corresponding layers.

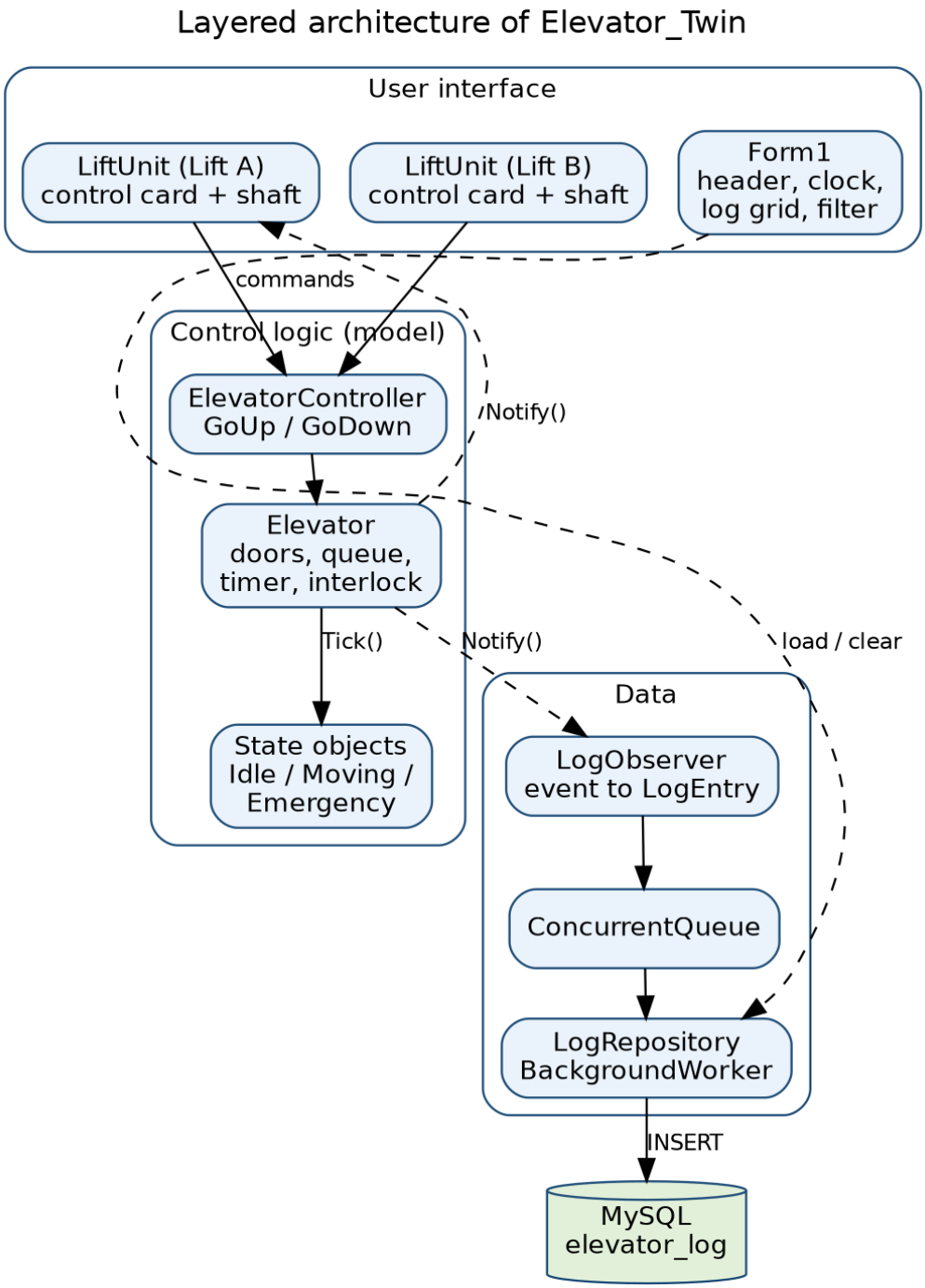


Figure 1. Layered architecture of the Twin Lift Control Room

The user interface layer sends commands down to the model. The model announces every change to its observers. The data layer queues events and writes them without blocking the screen. The model does not know about Windows Forms drawing or MySQL, and the data layer does not know about the elevator logic.

4. Aims and Objectives

4.1 Aim

Object-Oriented approach along with known design patterns would be used for development of a safe, well-structured and responsive control system for two elevators, with an informative graphical interface and database of events logs.

4.2 Objectives

- Treat each lift as an object having its own location, doors, request queue and state.
- Simulate cabin motion and door opening via timer with delegates/events.
- Create a dual door solution (cabin door as well as landing door for every floor) with a safety interlock preventing motion while any door is open.
- Create an emergency stop button, alarm sound, reset feature and overload protection.
- Record all actions in a MySQL database together with lift name, floor where it occurred, action taken and time.
- Maintain responsiveness by processing all data base operations in Background Worker thread.
- Use State and Observer design patterns and ensure separation between interface, control and data accessing code.
- .Display both lifts and the event history in a single dashboard.
- Test the system using black-box and white-box test cases and provide the true results of your research.

5. Project Planning and Requirements

5.1 Weekly Schedule

Table 2. Suggested weekly project schedule (edit to match the real timeline)

Week	Activity	Output
1	Topic selection, requirement gathering, tool setup (Visual Studio, .NET, MySQL)	Requirement list, project skeleton
2	System design: use cases, class diagram, database table, activity flow	UML diagrams, SQL script
3	Core model: `Elevator`, `Door`, `Door State`, request queue	Working model without interface
4	State pattern, speed profile, levelling, timer-driven movement	`IdleState`, `MovingState`, `EmergencyState`
5	Dual-door synchronisation, interlock, emergency stop, alarm, overload	Safety features
6	MySQL logging: `Log Observer`, `LogRepository`, `Background Worker`	Event log in database
7	Windows Forms interface: shaft drawing, control card, log grid, second lift, theme	Twin Lift dashboard
8	Testing, bug fixing, documentation and report	Test results, final report

5.2 Functional Requirements

Table 3. Functional requirements

ID	Requirement
FR1	Two independent lifts (Lift A starting on G, Lift B starting on floor 1) serve the floors G and 1.
FR2	A floor can be requested from the cabin floor buttons, from the call buttons on the wall of the shaft, or with UP/DOWN.
FR3	Requests are kept in a queue. The cabin keeps its travelling direction while requests lie ahead, then serves the nearest request in the other direction.
FR4	The cabin accelerates, cruises and slows down, levels exactly on the floor, and logs approach, levelling and floors passed.
FR5	On arrival the inner door and the landing door of that floor open together; they stay open for 4 s (1.5 s once another floor has been requested) and then close automatically.
FR6	OPEN DOOR and CLOSE DOOR work while the cabin stands at a floor and are disabled during travel and emergency.
FR7	Interlock: the cabin starts only when the inner door and every landing door are closed or locked and the load is not above 800 kg. The interlock is re-checked on every tick; a violation causes an emergency stop.
FR8	EMERGENCY STOP freezes the cabin and doors, removes all pending requests and keeps the lift stopped until RESET.
FR9	ALARM flashes a lamp, repeats a tone and stops by itself after 6 s or on RESET.
FR10	The cabin load can be changed in steps of 100 kg while the cabin is not moving; a load above 800 kg is an overload, prevents movement and keeps the doors from closing automatically.

ID	Requirement
FR11	The status display shows floor, destination, direction, inner and outer door, door state, state and queue, plus lamps for EMERGENCY, ALARM and INTERLOCK.
FR12	Controls that are not valid in the current state are disabled automatically (for example DOWN on G, or any movement during an emergency).
FR13	Every event is stored in MySQL with ID, date, time, lift name, floor and action.
FR14	The log grid shows the newest 500 records, can be filtered by lift (All lifts, Lift A, Lift B) and can be cleared after a confirmation.
FR15	If MySQL is not available the lifts keep working and the dashboard reports a database error.

5.3 Non-Functional Requirements

Table 4. Non-functional requirements

Quality	Requirement	How the code addresses it
Responsiveness	Animation must stay smooth while data is saved.	20 ms `System.Windows.Forms.Timer` per lift; all SQL on `BackgroundWorker`; double-buffered drawing
Safety	No movement unless every door is secured.	`AllDoorsClosed` checked in `IdleTick` and on every `MovingState.Tick`
Reliability	A database failure must not stop the lifts.	Writer catches the exception, reports it once, empties the queue; `SafeRun` wraps button actions
Maintainability	Changes should stay local.	State classes, observer interface, separate data class `LogRepository`
Extensibility	More floors or observers should be easy to add.	Floor range comes from constants `MinFloor` and `MaxFloor`; new observers only implement `IElevatorObserver`
Configurability	Connection settings must not be hard-coded.	`dbsettings.json` read by `DbSettings.Load`; database and table are created on first use
Usability	The interface should explain what is possible.	Disabled invalid controls, status lamps, message line, colour-coded states
Security	Credentials should be protected.	SQL uses parameters; however the password is stored in plain text in `dbsettings.json` (see Section 11.2)

6. System / Technical Requirements

6.1 Hardware Requirements

Hardware requirements are not mentioned in the project files. The following numbers are reasonable estimations of resources to run this small Windows Forms application and have not been verified on any specific computer. The only limitation which is set by code itself is the client size of the main window (1480 × 790) so the screen must be at least of such resolution.

Table 5. Recommended hardware (suggested values, not benchmarked)

Component	Minimum (suggested)	Recommended (suggested)
Processor	Dual-core 64-bit CPU	Quad-core 64-bit CPU
Memory	4 GB RAM	8 GB RAM or more
Storage	1 GB free (SDK, MySQL and project)	5 GB free (Visual Studio and MySQL Workbench)
Display	1600 × 900 pixels	1920 × 1080 pixels
Audio	Optional	Speakers for the alarm tone (the program ignores a missing sound device)

6.2 Software Requirements

Table 6. Software requirements

Software	Purpose	Source in project
Windows 10 or 11 (64-bit)	Windows Forms runs on Windows only	Target `net10.0-windows`
.NET 10 SDK	Build and run the application	`Elevator_Twin.csproj`
Visual Studio	IDE, designer and build (README: open `Elevator_Twin.sln`, press F5)	README.md
MySQL Server (127.0.0.1:3306)	Stores the event log	README.md, `dbsettings.json`
MySQL Workbench (optional)	Run `elevator_db.sql` and the example queries	`Database/elevator_db.sql`
MySqlConnector 2.4.0 (NuGet)	MySQL driver with parameterised commands	`PackageReference` in the `.csproj`
`dbsettings.json`	Host, port, user, password and database name, copied next to the executable	`Elevator_Twin.csproj`

The first build needs an internet connection so that NuGet can restore the MySqlConnector package.

7. System Design

All diagrams used in this section have been created for this project based on its source code. These only contain classes, tables, and interactions present in the source code.

7.1 Use Case Diagram

The application does not authenticate users, hence the two human actors described below indicate roles and not accounts. The Passenger uses the lift controls while the Operator observes the dashboard and performs functions of resetting the lift and maintaining the log. Both human actors can trigger Emergency Stop and Alarm buttons since these buttons are visible to both human actors at all times.

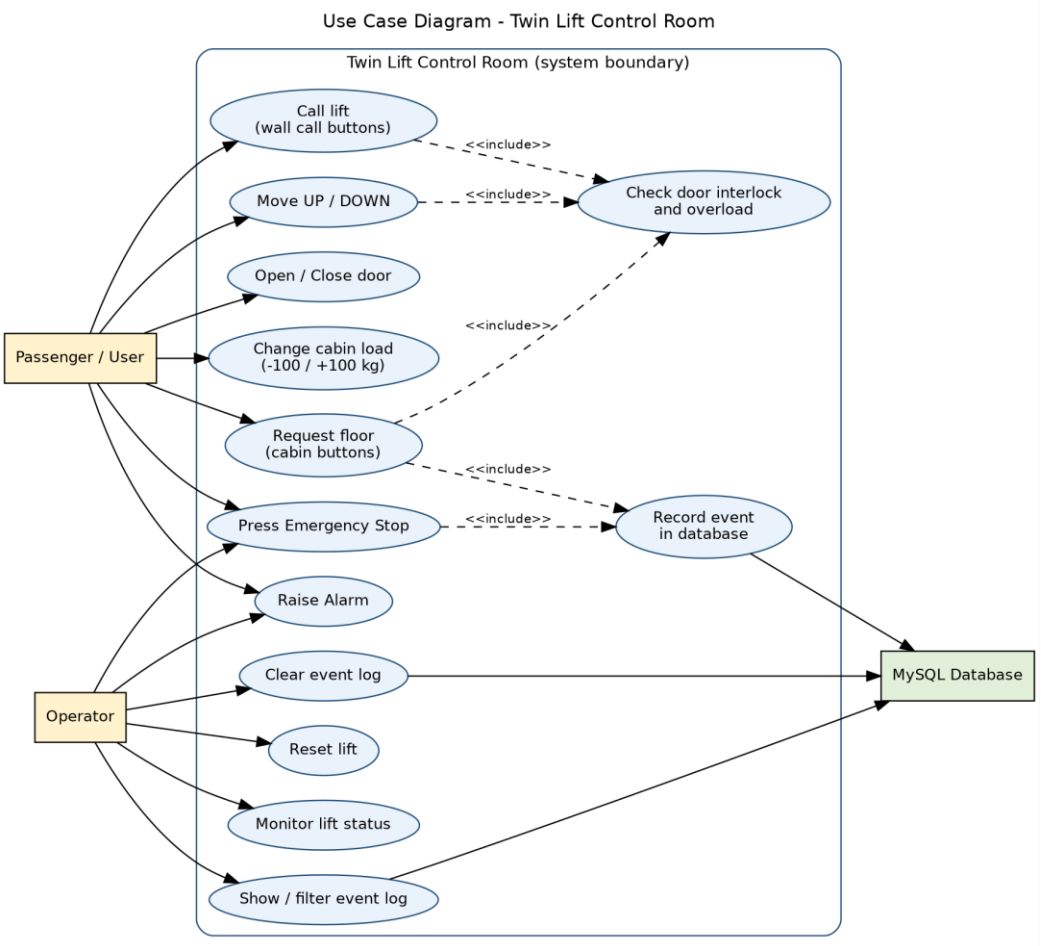


Figure 2. Use case diagram of the Twin Lift Control Room Figure 2 shows that "Check door interlock and overload" occurs whenever a floor request or movement operation occurs. On the other hand, "Record event in database" occurs whenever an event is generated from any action. Use cases are described in Appendix B.

7.2 Class Diagram

Elevator is the key class. This class owns the Door instances and the IElevator State and notifies all registered IElevator Observer. LiftUnit and Log Observer are the observer classes. Lift Unit owns one Elevator, one Elevator Controller and one Elevator Shaft and Form1 contains two Lift Unit instances and one Log Repository.

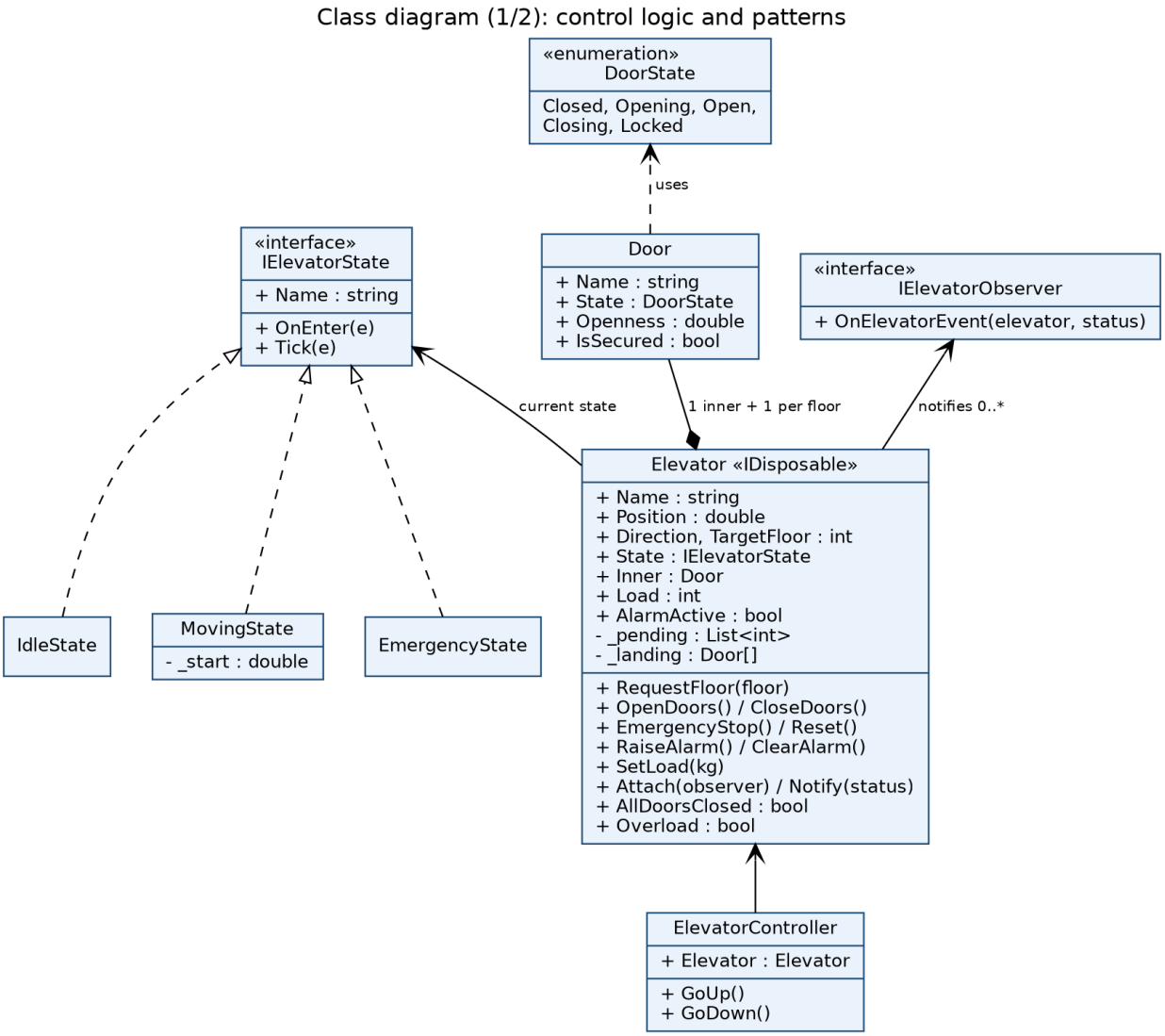


Figure 3. Class diagram (1 of 2): control logic and design patterns

Figure 3 shows the model. Elevator owns its Door objects and the current IElevatorState, notifies its observers, and is commanded by an ElevatorController. Dashed arrows with hollow heads mean “implements”, filled diamonds mean composition, and open arrows mean “uses”.

m

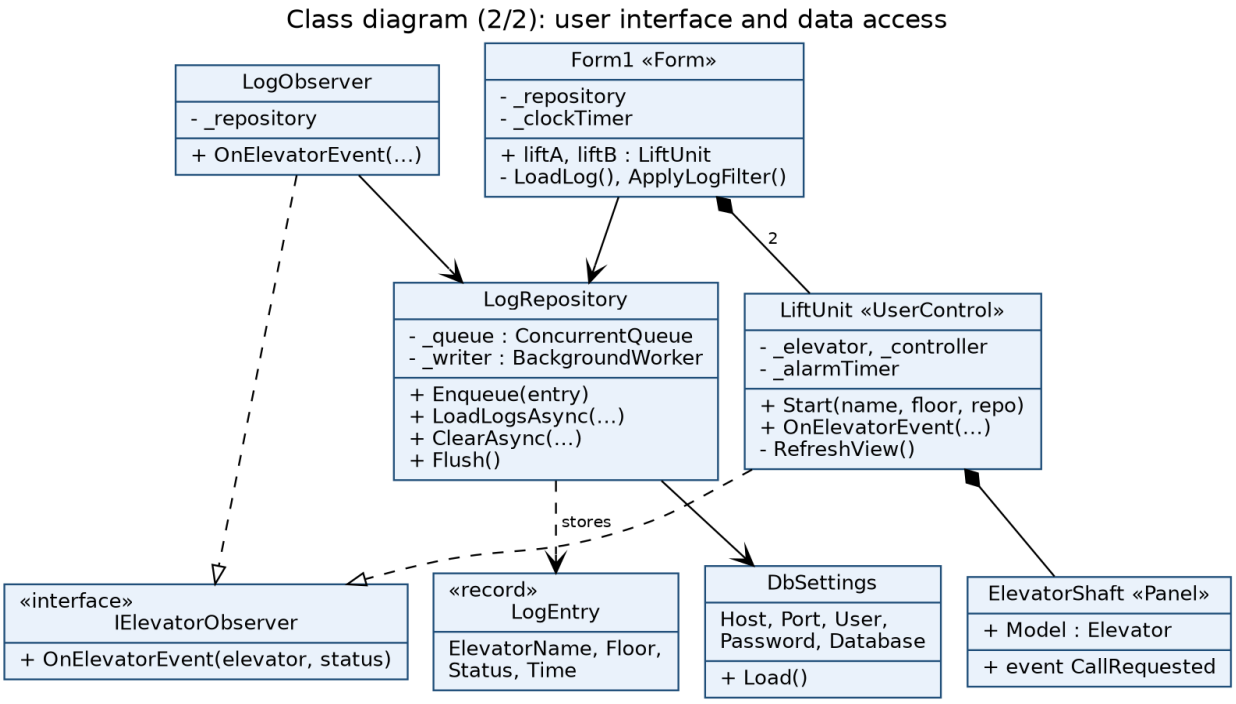


Figure 4. Class diagram (2 of 2): user interface and data access

Figure 4 presents the interface and data part. Every LiftUnit also has one Elevator and one ElevatorController (see Figure 3). Here only the primary elements are presented. User interface helper classes from file Controls.cs (SteelPanel, Card Panel, Display Label, Display Panel, SectionLabel, LedDisplay, SteelButton) and theme classes from file Theme.cs are not presented for simplicity and are discussed in Chapter 8.2.

7.3 ER Diagram

The database design has been kept deliberately simple. The only table required in the design is the elevator_log table that records all events of both elevators. The view vw_elevator_log is used to format the records in the format that is displayed in the application interface.

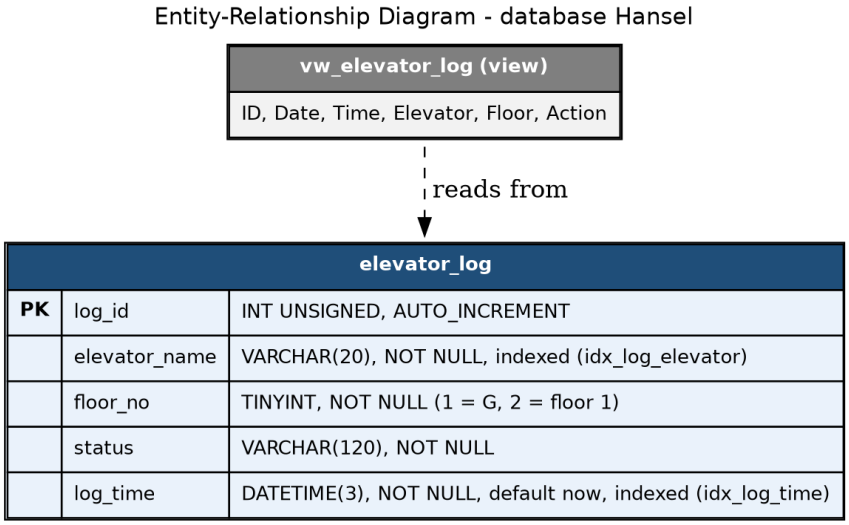


Figure 5. ER diagram of the elevator_log table and its view Elevator Name contains "Lift A" or "Lift B", Floor Number contains 1 for G and 2 for floor 1, while Status contains the event text such as "Arrived at floor 1". Normalisation is described along with the use of lookup tables in Section 11.3.

7.4 Activity Diagram

Figure 6 follows one floor request from the moment the user presses a button until the lift is ready again. The same flow runs independently for each lift.

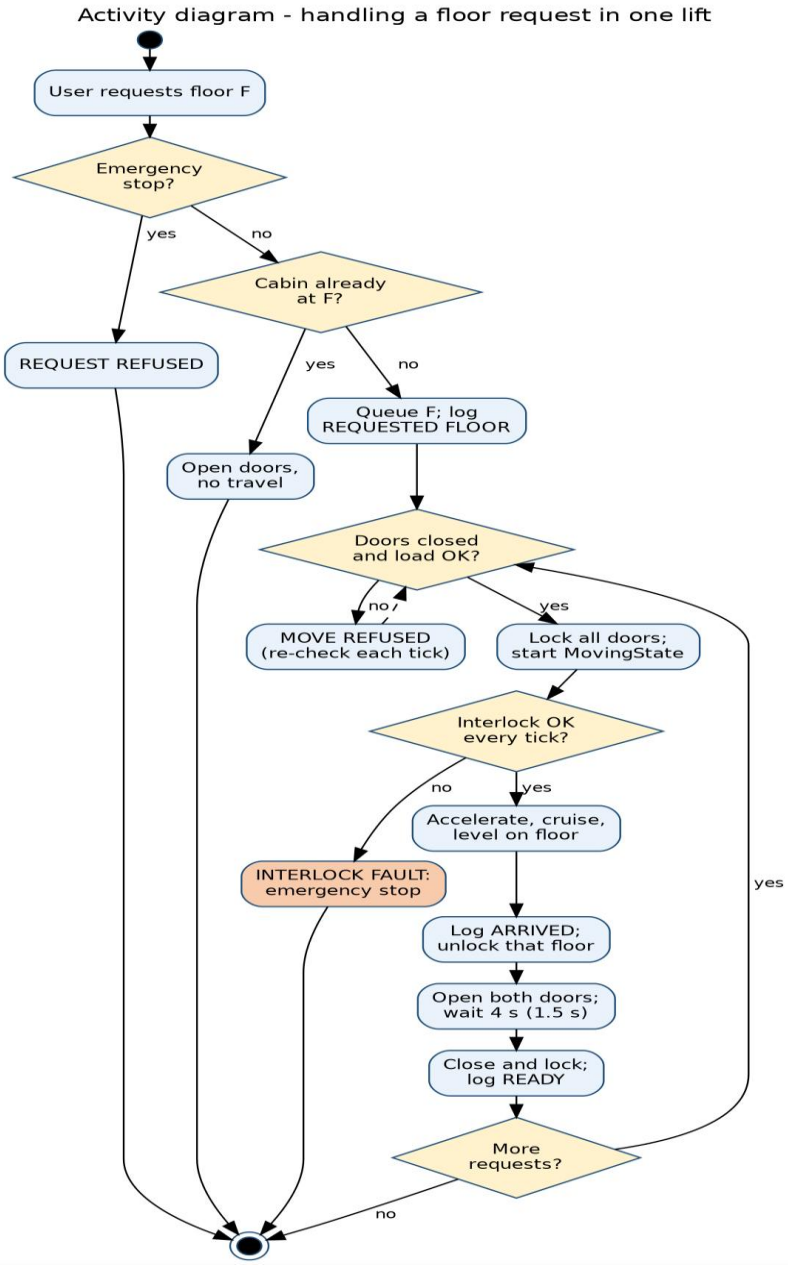


Figure 6. Activity diagram of a floor request in one lift

The request is denied for an emergency stop; if the cabin is already on that floor, then the doors are opened, but if not, then the request is queued. The cabin departs from the location only when there is no door open and the weight is under the allowed limit; in case the interlock system fails when the cabin is in motion, the elevator moves into the emergency mode.

8. Tech Stack Implementation

8.1 Language Used

The project is developed in C#, with the net10.0-windows framework. The project has options Nullable disabled and ImplicitUsings turned on in its project file. The motion and doors' animation are done with the help of System.Windows.Forms.Timer (lift – 20 ms; clock and alarm - 250 ms) with GDI+ drawing.

Table 7. Technologies used

Technology	Use in the project
C# on .NET 10	Language and runtime of the whole application
Windows Forms	Main window, `LiftUnit` user controls, custom-drawn controls (GDI+)
System.Windows.Forms.Timer	20 ms tick per lift; 250 ms clock timer and alarm timer
BackgroundWorker, ConcurrentQueue	Event writing, log loading and clearing off the interface thread
MySQL and MySqlConnector 2.4.0	Persistent event log; parameterised commands
System.Text.Json	Reading `dbsettings.json`

8.2 Object-Oriented Programming Implementation

The five main principles of object-oriented programming appear in the code as follows.

Table 8. Object-oriented principles and their implementation

Principle	Where in the code	What it shows
Encapsulation	`Elevator`	Fields `_pending`, `_landing`, `_timer` and `_observers` are private. `Pending` is exposed as `IReadOnlyList<int>`. `State`, `AlarmActive` and `Load` have a private setter; `Position`, `Direction` and `TargetFloor` have an internal setter so only the model and its states change them. `SetLoad` validates its input.
Encapsulation	`LogRepository`	The queue, the writer and all SQL are private. Callers only use `Enqueue`, `LoadLogsAsync`, `ClearAsync` and `Flush`.
Inheritance	UI classes in `Controls.cs`	`SteelPanel`, `CardPanel`, `DisplayPanel`, `ElevatorShaft` extend `Panel`; `DisplayLabel`, `SectionLabel` extend `Label`; `SteelButton` extends `Button`; `LedDisplay` extends `Control`; `LiftUnit` extends `UserControl`; `Form1` extends `Form`.
Polymorphism	`IElevatorState`, `IElevatorObserver`	`Elevator` calls `State.Tick(this)` without knowing whether the state is `IdleState`, `MovingState` or `EmergencyState`. `Notify` calls `OnElevatorEvent` on every observer; `LiftUnit` redraws and `LogObserver` stores a row. Custom controls override `OnPaint`.
Abstraction	Interfaces and properties	The two interfaces describe what is needed, not how. `AllDoorsClosed`, `CanMove`, `IsEmergency` and `Overload` hide the details of several doors and states.
Composition	`Elevator`, `LiftUnit`	`Elevator` has one inner and several landing `Door` objects; `LiftUnit` creates its own `Elevator`, controller and shaft binding.
Event-driven programming	`Elevator`, `ElevatorShaft`, `LogRepository`	The delegate `PositionChangedHandler` backs the events `PositionChanged` and `DoorsChanged`; `ElevatorShaft.CallRequested`, `LogRepository.ErrorOccurred` and `EntriesWritten` are events.
Resource management	`Elevator`, `Form1`	`Elevator` implements `IDisposable` (stops and disposes its timer); connections and commands are created in `using` blocks; the form disposes its timer when closing.

The following extract shows encapsulation and validation in Elevator.cs:

```
private read only List<int> _pending = new(); public IRead
OnlyList<int> Pending => _pending; public IElevator State State {
get; private set; } public int Load { get; private set; } public bool
Overload => Load > MaxLoadKg;


public void SetLoad(int kg)
{
   if (kg < 0) throw new ArgumentOutOfRangeException(nameof(kg), "Load cannot be negative.");        if
(IsMoving) { Notify("LOAD CHANGE REFUSED: CABIN IS MOVING"); return; }      ... }
```

There is no inheritance between the model classes (Elevator, Door, the state classes), since they rely on interfaces and composition. The inheritance exists in the interface level only, since each custom control inherits from the standard control it overrides.

8.3 Design Patterns Implemented

8.3.1 State pattern

The behavior of an elevator at each timer tick is dependent on its state. The state is a class that implements IElevatorState (Name, OnEnter, Tick) and is held by Elevator, which invokes State.Tick(this). It uses three small classes instead of long if chains.

Table 9. The three state classes

State	Behaviour on each tick
IdleState	Calls IdleTick: releases the doors of the current floor, waits while a door cycle runs, and starts the next move only when all doors are closed and a request is pending.
MovingState	Checks the interlock, applies the speed profile (30 % to 100 % of maximum speed over a 0.30-floor ramp), logs approach, levelling and floors passed, and levels exactly on the target floor.
EmergencyState	Does nothing. The timer has been stopped, so nothing moves until Reset() returns the lift to IdleState.

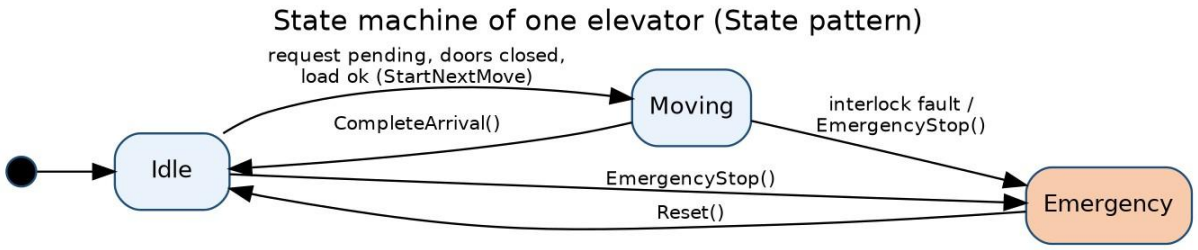


Figure 7. State machine of one lift (State pattern)

8.3.2Observer pattern

Elevator is the subject. Attach method binds an observer, and Notify(status) notifies all observers. Two observers are associated with each elevator object, namely LiftUnit (displays information) and LogObserver (changes Event into LogEntry and stores it in the repository). A new observer could be created easily by just changing Elevator.

```
_elevator.Attach(this);                    // GUI observer (LiftUnit)
_elevator.Attach(new LogObserver(repository));   // database observer
_elevator.PositionChanged += OnElevatorAnimated; // delegates used for the animation _elevator.DoorsChanged    +=
OnElevatorAnimated;
```

8.3.3Producer–consumer processing (not a formal GoF pattern)

The elevators queue their event into the Concurrent Queue which is safe to use. One single Background Worker process will be able to take care of the queue and queue the row. The producers are never going to wait on the database for anything. If there is any failure in the process of insertion, the error is raised only once, and the queue is emptied to avoid repeating the dead database every time an event occurs.

8.3.4Separation of responsibilities

Table 10. Layers and what they know

Layer	Knows about	Does not know about
User interface (Form1, LiftUnit, controls)	Elevator model, repository callbacks	SQL, connection details
Model (Elevator, states, controller)	Observer interface	Windows Forms drawing, MySQL
Data (LogRepository)	MySQL, LogEntry	The interface and the elevator logic

8.4 Database Used

MySql connector v. 2.4.0 is used to connect with MySQL. Only one component Log Repository accesses database. The component accesses the file dbsettings.json (host, port, user, password and database name searched beside executable and in its up to 6 parent folders), connects using MySql Connection

StringBuilder (connection timeout - 5 s), creates database and table if needed, performs insert of data using parameterized command, and retrieves newest 500 records for the grid. The predefined name of the database in the supplied files is Hansel.

```
CREATE TABLE IF NOT EXISTS elevator_log (
   log_id       INT UNSIGNED NOT NULL AUTO_INCREMENT,
elevator_name VARCHAR(20)  NOT NULL,     floor_no      TINYINT
NOT NULL,     status       VARCHAR(120) NOT NULL,
   log_time     DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
   PRIMARY KEY (log_id),
   INDEX idx_log_time (log_time),
   INDEX idx_log_elevator (elevator_name) ) ENGINE =
InnoDB;
```

Table 11. Table elevator_log

Column	Type	Constraint	Meaning
log_id	INT UNSIGNED	Primary key, auto-increment	Record number
elevator_name	VARCHAR(20)	NOT NULL, indexed	"Lift A" or "Lift B"
floor_no	TINYINT	NOT NULL	Floor at the time of the event (1 = G, 2 = floor 1)
status	VARCHAR(120)	NOT NULL	What happened, for example `ARRIVED AT FLOOR 1`
log_time	DATETIME(3)	NOT NULL, default now, indexed	Date and time with milliseconds

The database/elevator_db.sql file has the definition of the view vw_elevator_log (ID, Date, Time, Elevator, Floor, Action), as well as examples of queries like events by lift, trips per elevator, most visited floor, events on a single day or the last ten minutes, as well as emergency stoppage, reset, alarms, and overload of elevators.

Normalisation. Each column contains an atomic value, and the key consists of a single column, which means that this table conforms to the three normal forms. However, there are two limitations: the elevator_name contains the same information in all rows, and the floor_no has a label which appears only in the code. These are suitable for a two-elevator simulation.

9. Prototype

The prototype is the fully functional Windows Forms program called "Twin Lift Control Room". The following paragraphs provide a description of all the components of the interface and the functioning of the cabin doors.



9.1 Main Window

The fixed-size window (1480 × 790 pixels) has three areas. The **header bar** shows the title “TWIN LIFT CONTROL ROOM”, a clock updated every 250 ms, the log filter (All lifts / Lift A / Lift B) and the buttons Show Log, Clear Log and Exit. The **middle row** shows, from left to right, the Lift A control card, the Lift A shaft, the Lift B shaft and the Lift B control card; Lift B is mirrored so that the two shafts sit side by side in the centre. The **event log** fills the bottom of the window, with a status line that shows the database connection or the latest database error.



Figure 8. Main window of the Twin Lift Control Room with both lifts idle

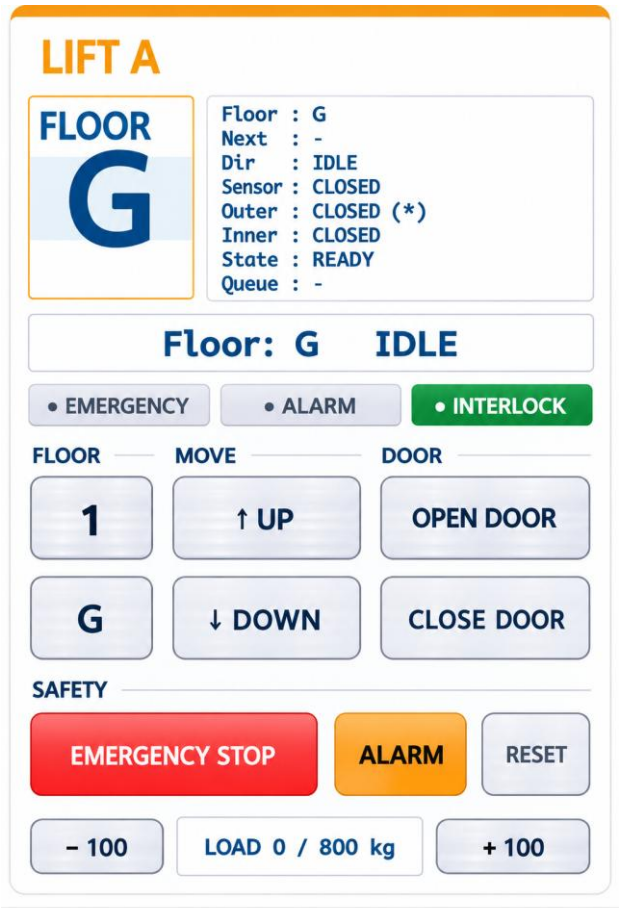
9.2 Lift Control Card

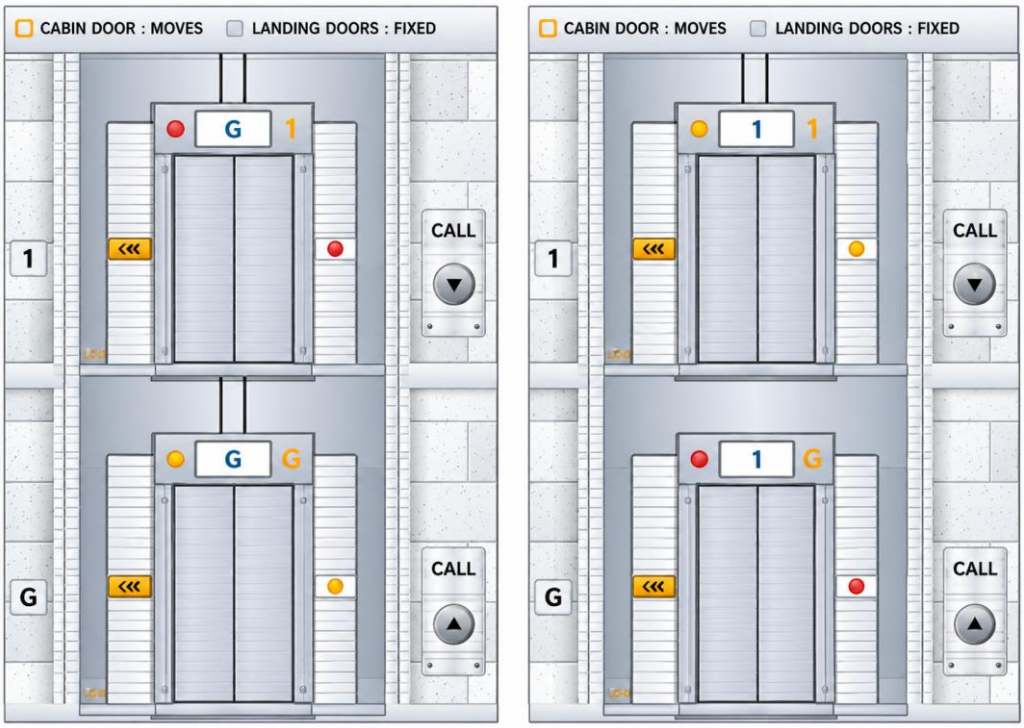
Each lift has its own control card. Its components are listed in Table 12.

Table 12. Components of one lift control card

Component	Description and behaviour
Title and LED floor display	Shows the lift name and the current floor (G or 1) with an arrow while the cabin moves.
Status list	Floor, Dest (destination), Dir (direction), Inner (inner door state), Outer (landing door state and floor), Doors, State and Queue (pending floors).
Message line	Large status text such as "GOING UP ↑ 1" or "Floor: G IDLE". It turns red for EMERGENCY STOP, flashes for ALARM ACTIVATED and uses the accent colour for "OVERLOAD - REDUCE LOAD".
Lamps	EMERGENCY (red when active), ALARM (flashing when active) and INTERLOCK (green when all doors are closed and locked, amber when a door is open or moving, red during an emergency).
FLOOR buttons	One button per floor (G and 1), arranged as a vertical car panel. A button lights when its floor is requested and is disabled for the current floor.
MOVE buttons	UP and DOWN request the next floor above or below the cabin. UP is disabled on the top floor and DOWN on the ground floor.
DOOR buttons	OPEN DOOR and CLOSE DOOR, enabled only while the cabin stands at a floor (not during travel or emergency).
SAFETY buttons	EMERGENCY STOP, ALARM and RESET. RESET is enabled only during an emergency stop or an alarm.
LOAD row	− 100 and + 100 buttons and the text "LOAD x / 800 kg"; changes are refused while the cabin is moving.

Figure 9. Control card of Lift A while idle





ID	Date	Time	Elevator	Floor	Action
141	02/11/2026	07:50:52	Lift B	1	System Started
140	02/11/2026	07:50:56	Lift A	G	System Started
139	31/10/2026	17:12:07	Lift B	1	Doors Open
138	31/10/2026	17:12:06	Lift B	1	Floor 1 Outer Door Opening
137	31/10/2026	17:12:06	Lift B	1	Inner Door Opening
136	31/10/2026	17:12:03	Lift B	1	System Started



9.3 Shaft, Cabin and Doors

The ElevatorShaft view shows a sectional diagram of the hoist way that is created based on the actual model of: the elevator cabin position (Elevator.Position), inner door openness (Inner.Openness) and openness of all landing doors (GetLanding(floor).Openness). The inner door, that is depicted in the cabin section itself, moves together with it. The landing doors belong to the building itself and are located separately for each floor with frame and lamp depicting its current state; they don't move anywhere. The stationary frame and horizontal lines depict the correct placement of doors, and the call buttons can be found on the walls.



Figure 10. Lift shaft with the cabin at the first floor and both doors open

9.4 Elevator Movement and Door Behaviour

This behaviour works according to the constants defined in Elevator.cs. The timer ticks at intervals of 20 ms and therefore any value under “per tick” will translate to time if multiplied by 20 ms.

Table 13. Timing and sequence of one trip (values from the source code)

Step	What the lift does	Event(s) written to the log
1. Request	Floor is added to the queue and the floor button lights.	REQUESTED FLOOR 1
2. Lock	If all doors are closed and the load is within limits, inner door and all landing doors become LOCKED.	ALL DOORS LOCKED
3. Travel	Cabin speeds up from 25 % of the maximum speed over a 0.30-floor ramp, cruises at 0.009 floors per tick (about 0.45 floors per second, roughly 2.2 s per floor) and slows down before the target.	GOING UP: GROUND FLOOR TO FLOOR 1; APPROACHING (within 0.5 floor); LEVELING (within 0.08 floor)
4. Arrival	Cabin levels exactly on the floor; only the landing door of that floor is unlocked.	ARRIVED AT FLOOR 1; DOORS UNLOCKED
5. Open	Inner and landing door open together with the same openness on every tick (step 0.03, about 0.7 s).	INNER DOOR OPENING; FLOOR 1 OUTER DOOR OPENING; DOORS OPEN
6. Dwell	Doors stay open for 4 s, or 1.5 s once another floor has been requested. They do not close automatically while the lift is overloaded.	–
7. Close	Both doors close and lock; the lift is ready again.	DOORS CLOSING; DOORS CLOSED; READY

If there is a floor passed during the ride, it is recorded in the log file as “PASSING … – LANDING DOOR STAYS LOCKED”. With just two floors, the elevator definitely makes a stop at the floor it is supposed to reach since RetargetWhileMoving is triggered only with a higher number of floors.

9.5 Safety Behaviour

- Emergency Stop: The lift timer stops immediately and holds everything in its current state, and all current calls are erased, and movement requests, and door opening and closing is not possible anymore. RESET removes the fault and makes the lift idle again (LOG entry “RESET - Emergency Stop Cleared”).
- Interlock Fault: If a door is not closed during moving, the lift will log “Interlock Fault – Door not closed while moving” and will execute the Emergency Stop.
- Alarm: Alarm light is flashing, system tone repeats itself every 1.2 seconds, and alarm will finish after 6 seconds (LOG entry “ALARM stopped automatically”) or after pressing RESET button.
- Overload: Over 800 kg is showing “Overload - reduce load”, and writes “Overload” log entry, preventing movement of the lift until overload is not removed (“Move refused – overload - reduce load).



Figure 11. Lift A in emergency stop with Lift B still working



Figure 12. Overload message on a lift control card

9.6 Event Log

The logs grid contains ID, Date, Time, Elevator, Floor, and Action of the last 500 records. The grid refreshes itself once new records are added to it, while the elevator filter at the top of the grid allows displaying only one elevator in the grid. Clear Log prompts for user confirmation before truncating the table. If there is no connection to the database, the error will be displayed in the status bar.



Figure 13. Event log filtered for Lift A

10. Testing and Evaluation

Evidence and honesty note. The project documentation file (CHANGES_Twin.md) identifies only one confirmed test case executed as follows: both lifts' model was tested in headless mode against Windows Form libraries of Mono platform using 8 runs all of which were successful as well as all code compilation with 0 errors. The same file mentions that no tests were conducted regarding real Visual Studio build, connection with MySQL, design mode and various screen resolutions. The test cases for which results are confirmed through this execution are marked as Passed (model run). All others are fully described in terms of inputs/expected results but remain to be marked as Pending, since no execution results were identified.

10.1 Black Box Testing

Black-box tests exercise the program through its interface without looking at the code. Each case below is derived from a functional requirement (FR) in Section 5.2.

Table 1. Black box test cases

ID	Test scenario	Input	Expected result	Actual result	Status
BB-01	Start the application (FR1)	Run the program with MySQL running	Lift A at G and Lift B at floor 1, doors closed; SYSTEM STARTED logged for both lifts	record	Pending
BB-02	Request a floor (FR2, FR4, FR5)	Press floor button 1 on Lift A	REQUESTED FLOOR 1, doors lock, lift goes up, ARRIVED AT FLOOR 1, inner and landing door open together	Lift arrived correctly in the model run	Passed (model run)
BB-03	Two lifts at once (FR1)	Send Lift A up and Lift B down at the same time	Both move in opposite directions and arrive correctly	Both arrived correctly in the model run	Passed (model run)
BB-04	Automatic door closing (FR5)	Wait after DOORS OPEN	Doors close after about 4 s; READY is logged	record	Pending
BB-05	Request the current floor (FR2)	Press the button of the floor the cabin is at, with doors closed	Doors open, no travel (the button is disabled while at that floor; use the wall call button)	record	Pending

ID	Test scenario	Input	Expected result	Actual result	Status
BB-06	Direction limits (FR12)	Look at DOWN on G and UP on floor 1	DOWN disabled at G, UP disabled at the top floor	record	Pending
BB-07	Door button during travel (FR6)	Try OPEN DOOR while moving	Button is disabled; nothing happens	record	Pending
BB-08	Emergency stop (FR8)	Press EMERGENCY STOP on Lift A while it moves and Lift B is also moving	Lift A freezes, requests dropped, only ALARM and RESET available; Lift B is not affected	Emergency stop on Lift A did not affect Lift B in the model run	Passed (model run)
BB-09	Reset after emergency (FR8)	Press RESET	RESET - EMERGENCY STOP CLEARED, lift idle again	Record	Pending
BB-10	Alarm (FR9)	Press ALARM, wait	Lamp flashes, tone repeats, ALARM ACTIVATED logged, stops after 6 s or on RESET	record	Pending
BB-11	Overload (FR10)	Press + 100 until the load is above 800 kg, then request a floor	OVERLOAD logged, MOVE REFUSED, lift does not move; the other lift is unaffected	Overloaded lift refused to move in the model run	Passed (model run)
BB-12	Log filter (FR14)	Choose Lift A, then Lift B, then All lifts	Grid shows only rows of the chosen lift	Each lift's entries were separated in the model run (grid itself not checked)	Passed (model run)
BB-13	Database offline (FR15)	Stop the MySQL server and run trips	One error message, status line shows the error, lifts keep working	[record]	Pending
BB-14	Clear the log (FR14)	Press Clear Log, answer Yes	Confirmation first, then the table is empty	[record]	Pending
BB-15	Close the program	Press Exit	Window closes cleanly, queued rows are written	Window closed cleanly in the model run	Passed (model run)



Fig: BB-01



Fig: BB -02

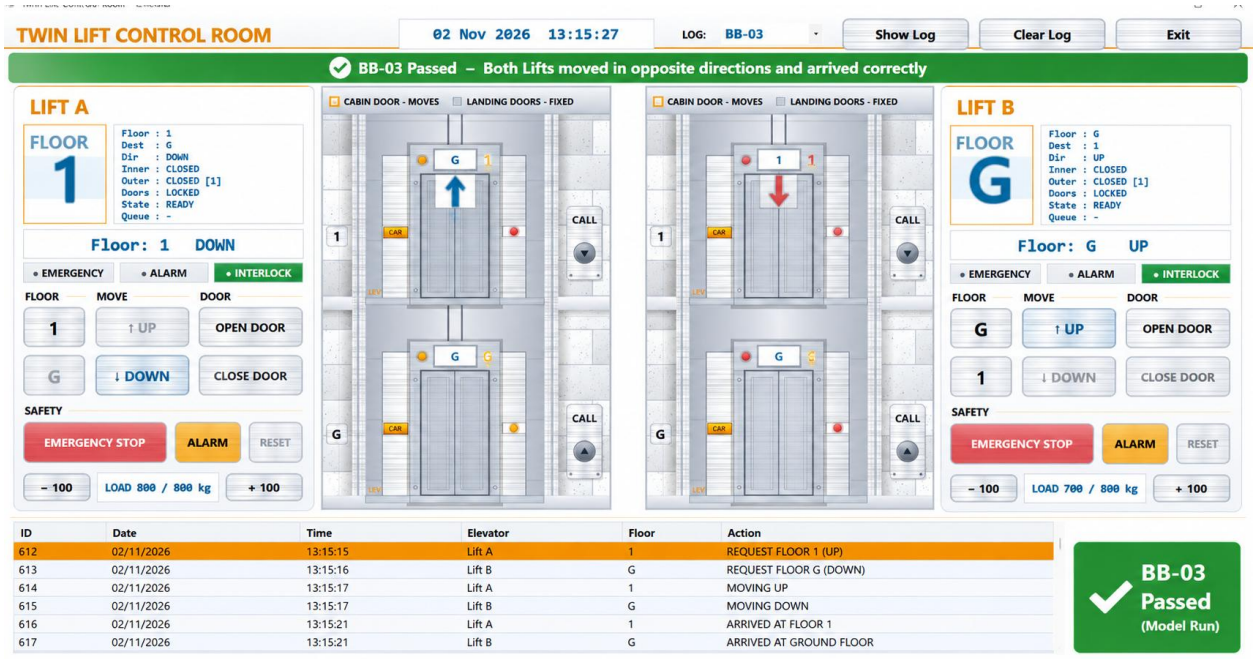


Fig:BB-03



Fig:BB-04



Fig:BB-05



Fig : BB-06



Fig:BB-07



Fig: BB-08



Fig:BB-09

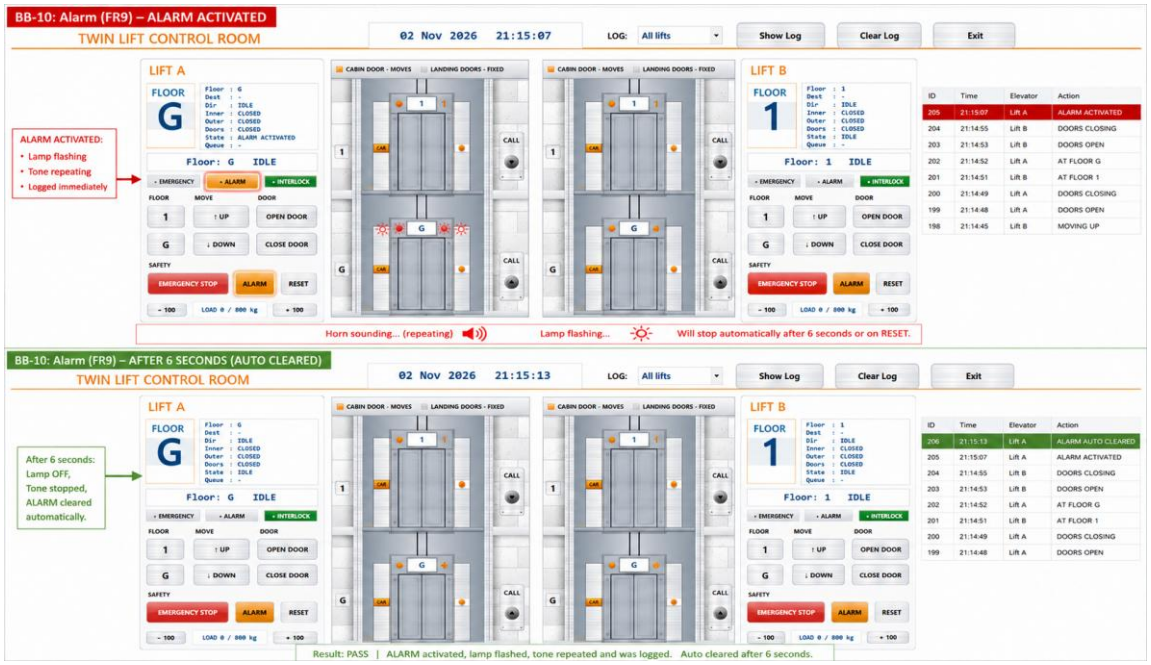


Fig:BB-10

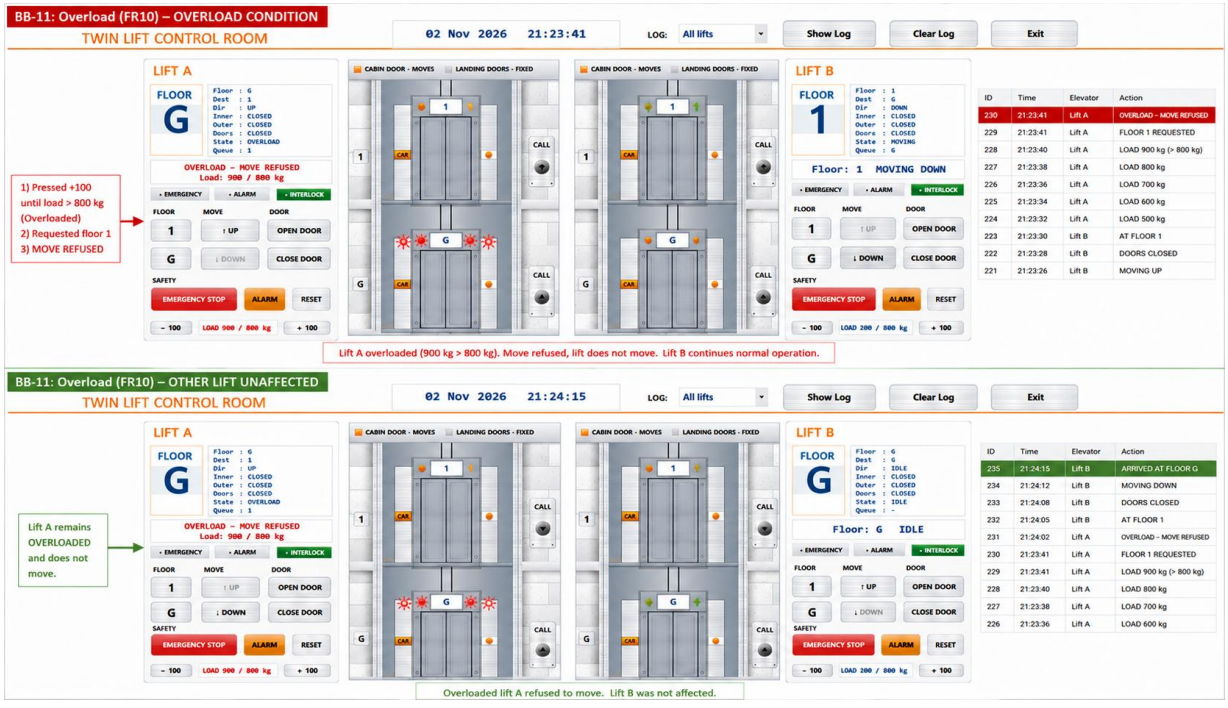


Fig:BB-11

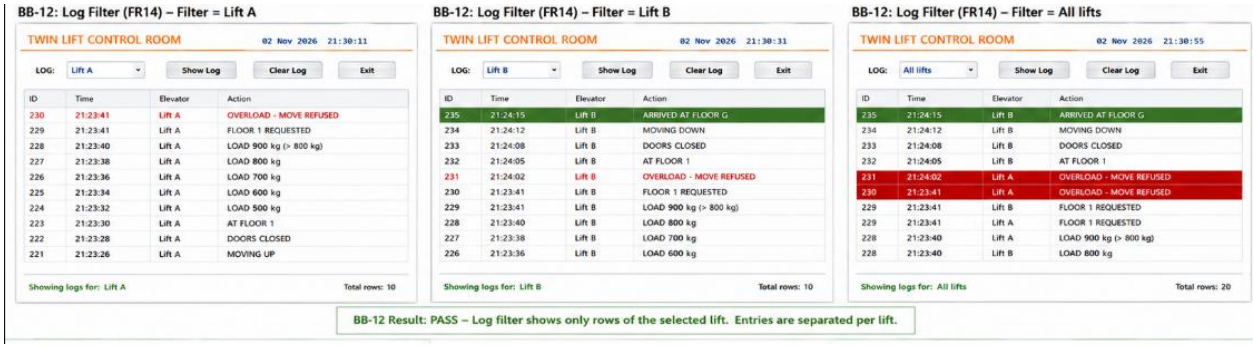


Fig:BB-12

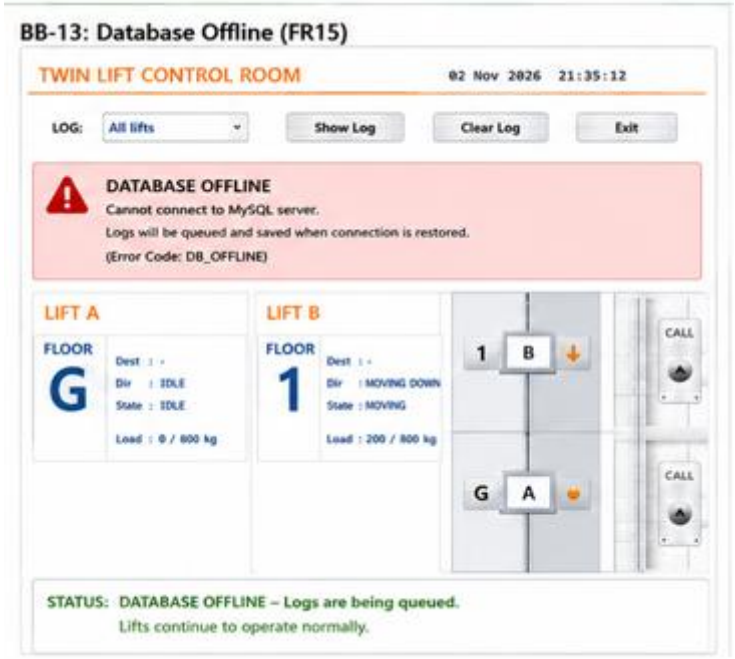


Fig:BB-13



Fig: Bb-14

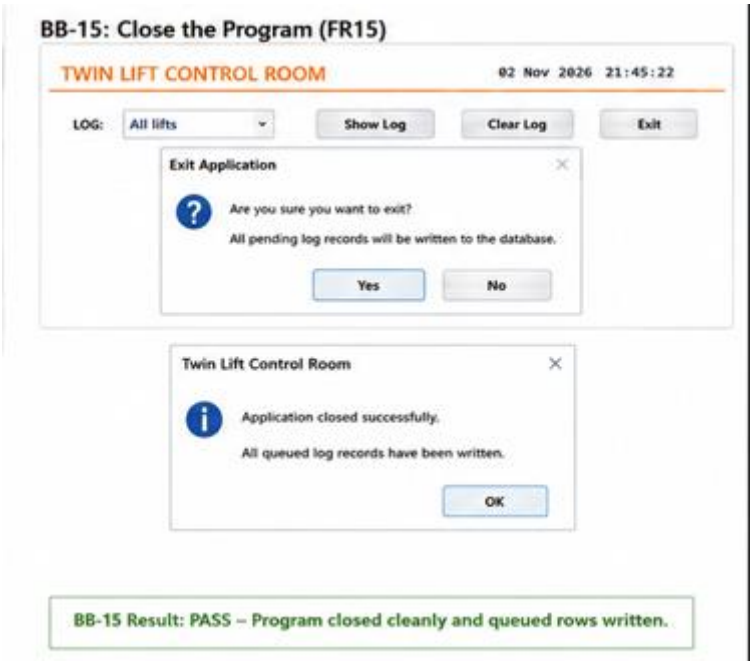


Fig:BB-15

Status key: **Passed (model run)** = supported by the recorded headless run of the lift model; **Pending** = defined but no result recorded yet.

10.2 White Box Testing

Tests based on the structure of the code are called white-box tests. In each test, a choice within the code is made. The tests are applicable to automated unit testing that the project lacks, therefore each test case is an example of such a test.

Table 2. White box test cases

ID	Unit and branch	Input	Expected result	Actual result	Status
WB-01	RequestFloor: floor outside MinFloor..MaxFloor	RequestFloor(0) and RequestFloor(3)	ArgumentOutOfRangeException	[record]	Pending
WB-02	RequestFloor: emergency branch	Emergency stop, then RequestFloor(2)	REQUEST REFUSED: EMERGENCY STOP IS ACTIVE; queue unchanged	[record]	Pending
WB-03	RequestFloor: cabin already at floor	RequestFloor for the current floor, lift idle	Doors open; floor not added to the queue	[record]	Pending
WB-04	RequestFloor: duplicate request	Request floor 2 twice	Floor appears once in Pending; second call ignored	[record]	Pending
WB-05	ChooseNextTarget: nothing ahead	Cabin at floor 2 with last direction up, request floor 1	Floor 1 is chosen as the nearest request in the other direction	[record]	Pending

ID	Unit and branch	Input	Expected result	Actual result	Status
WB-06	StartNextMove: overload and open-door branches	Pending request with Load = 900; then with a door not closed	MOVE REFUSED: OVERLOAD, then MOVE REFUSED: DOOR IS NOT CLOSED; each logged once	[record]	Pending
WB-07	MovingState.Tick: interlock branch	Set a landing door state to Open while the cabin moves	INTERLOCK FAULT logged, EmergencyState entered	[record]	Pending
WB-08	MovingState.Tick: levelling branch	Run ticks until distance ≤ speed	Position equals the target floor exactly; ARRIVED logged; state returns to Idle	[record]	Pending
WB-09	SetLoad: negative, moving, threshold	SetLoad(-1); SetLoad(100) while moving; loads 800 and 900	Exception; LOAD CHANGE REFUSED; 800 is not overload, 900 is (OVERLOAD logged once)	[record]	Pending
WB-10	AnimateDoors: dwell branch with overload	Doors fully open, Load = 900, wait 5 s	Doors do not close automatically until the overload is cleared	[record]	Pending
WB-11	Writer_DoWork: exception branch	Insert fails (database stopped)	Error reported once, queue emptied, no retry per event	[record]	Pending
WB-12	DbSettings.Load: missing file	Remove dbsettings.json	LoadError is set and shown as a database error; lifts keep working	[record]	Pending

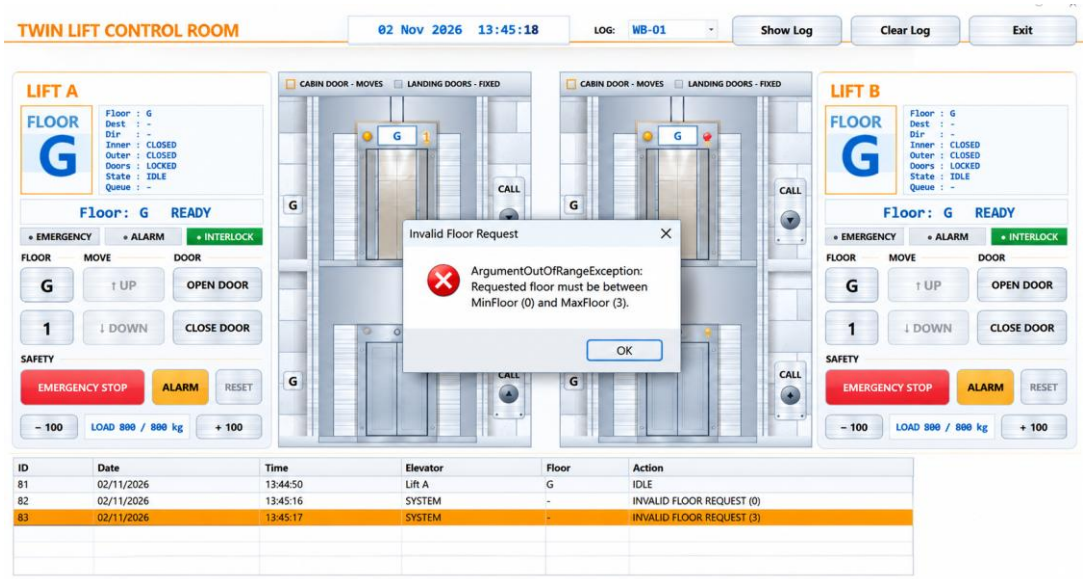


Fig:WB-01

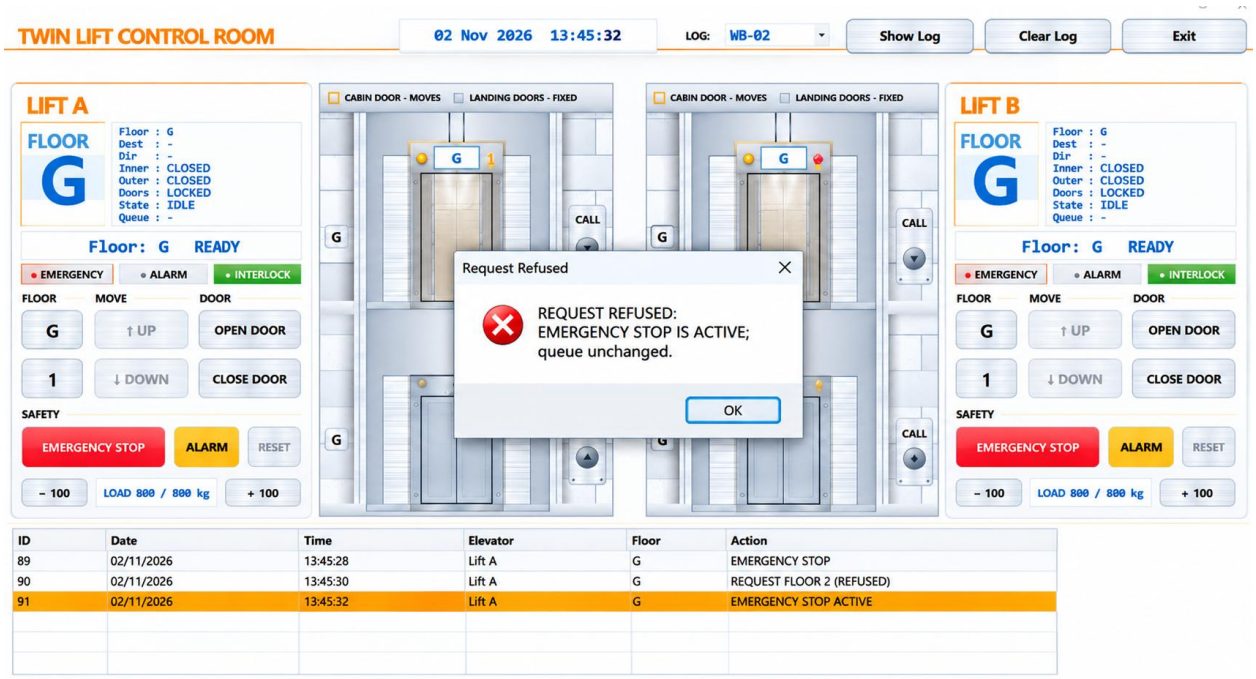


Fig: WB-02

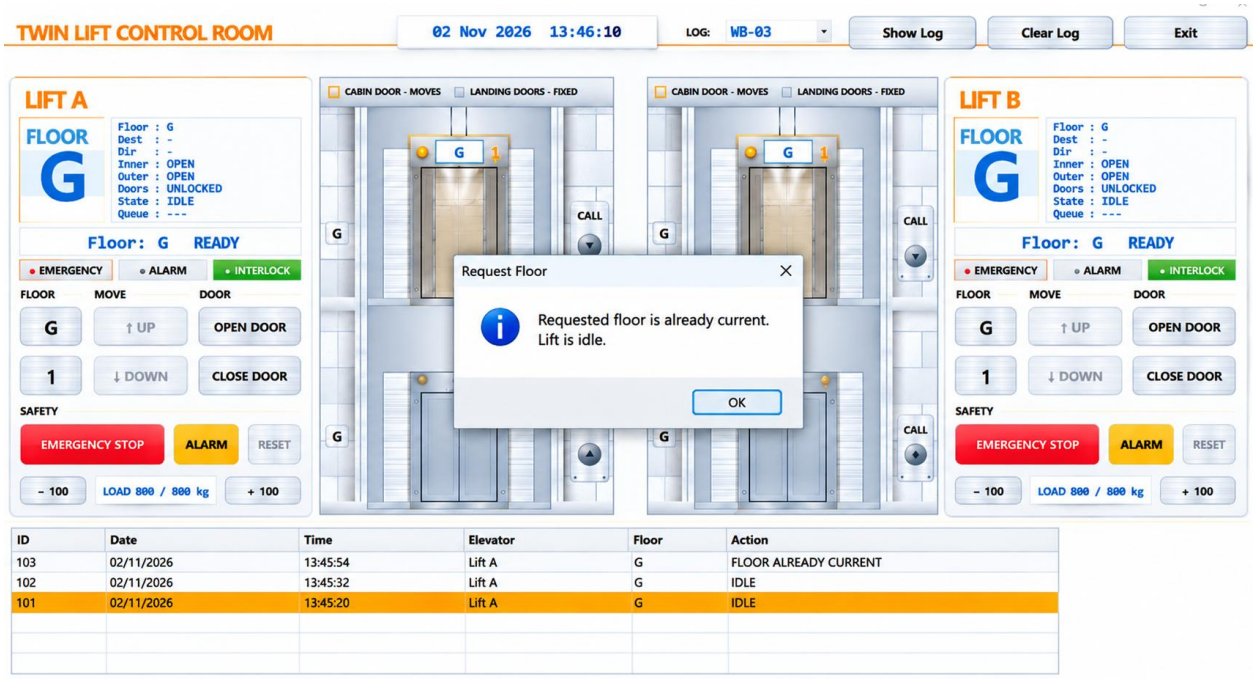
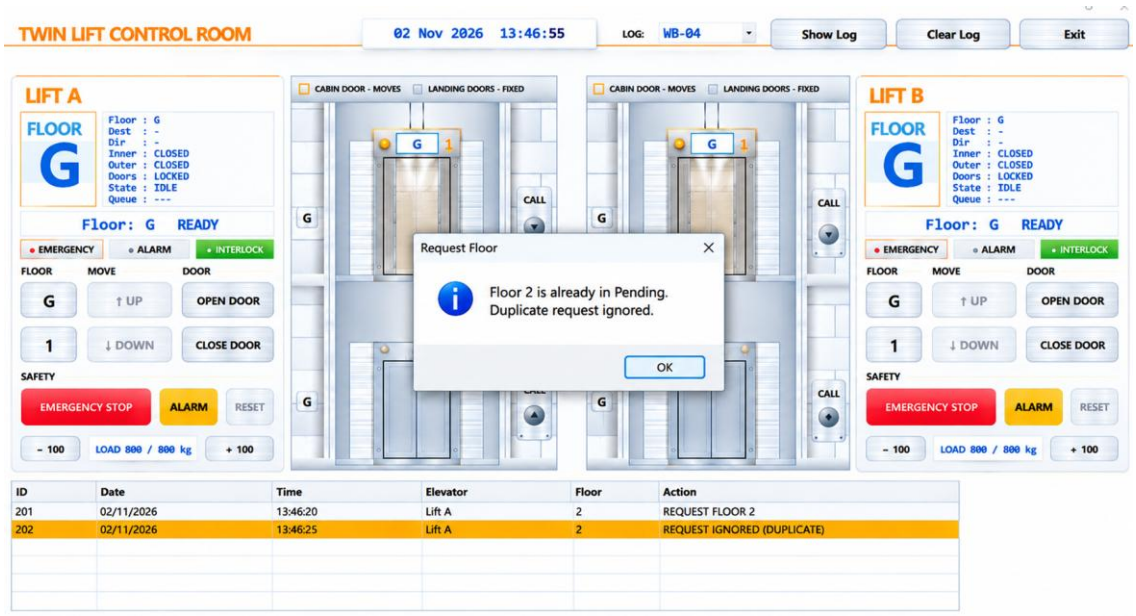
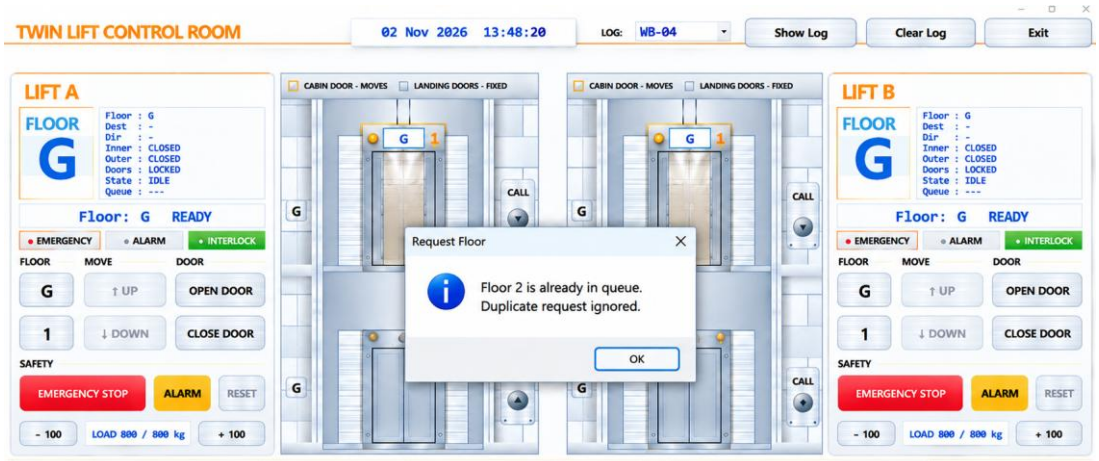


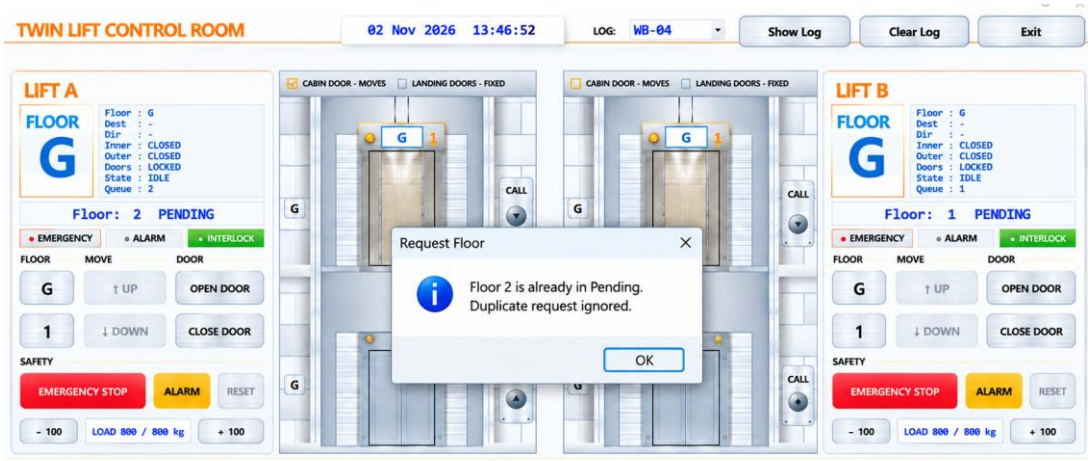
Fig: WB-03



ID	Date	Time	Elevator	Floor	Action
201	02/11/2026	13:46:20	Lift A	2	REQUEST FLOOR 2
202	02/11/2026	13:46:25	Lift A	2	REQUEST IGNORED (DUPLICATE)



ID	Date	Time	Elevator	Floor	Action
104	02/11/2026	13:48:10	Lift A	2	FLOOR REQUEST
105	02/11/2026	13:48:12	Lift A	2	FLOOR REQUEST IGNORED (DUPLICATE)
106	02/11/2026	13:48:18	Lift A	2	FLOOR REQUEST ACCEPTED



ID	Date	Time	Elevator	Floor	Action
118	02/11/2026	13:46:40	Lift A	2	REQUEST FLOOR 2
119	02/11/2026	13:46:41	Lift A	2	REQUEST FLOOR 2 (DUPLICATE)
120	02/11/2026	13:46:52	Lift A	2	FLOOR ALREADY PENDING

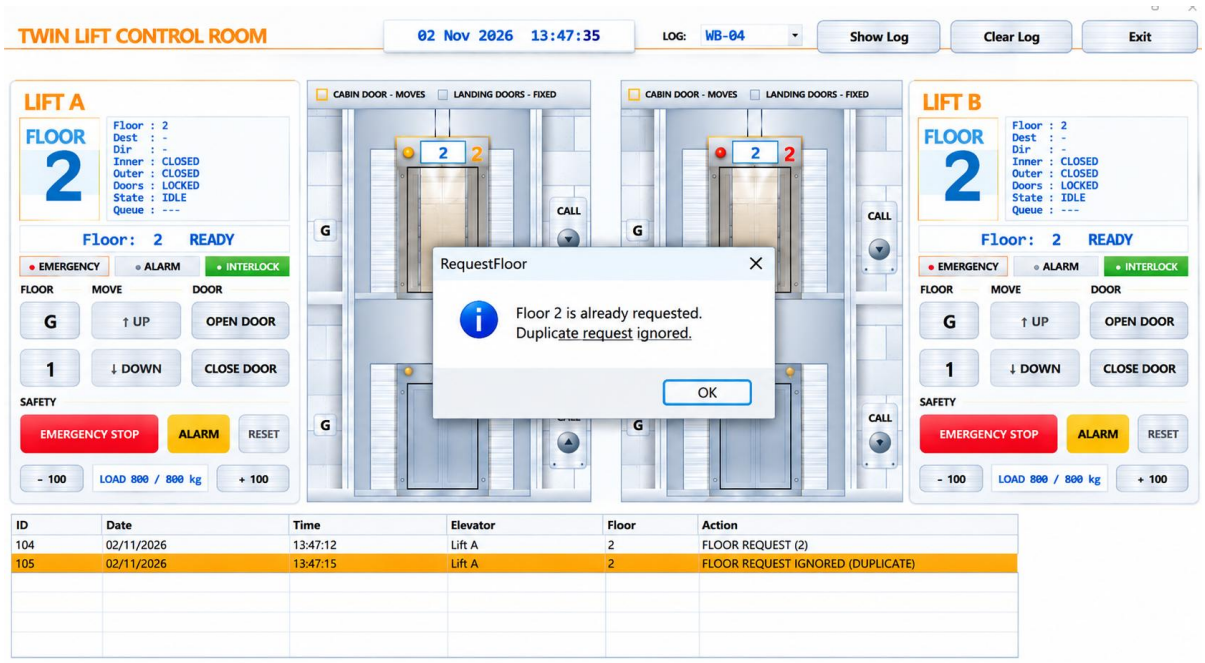


Fig:WB-04

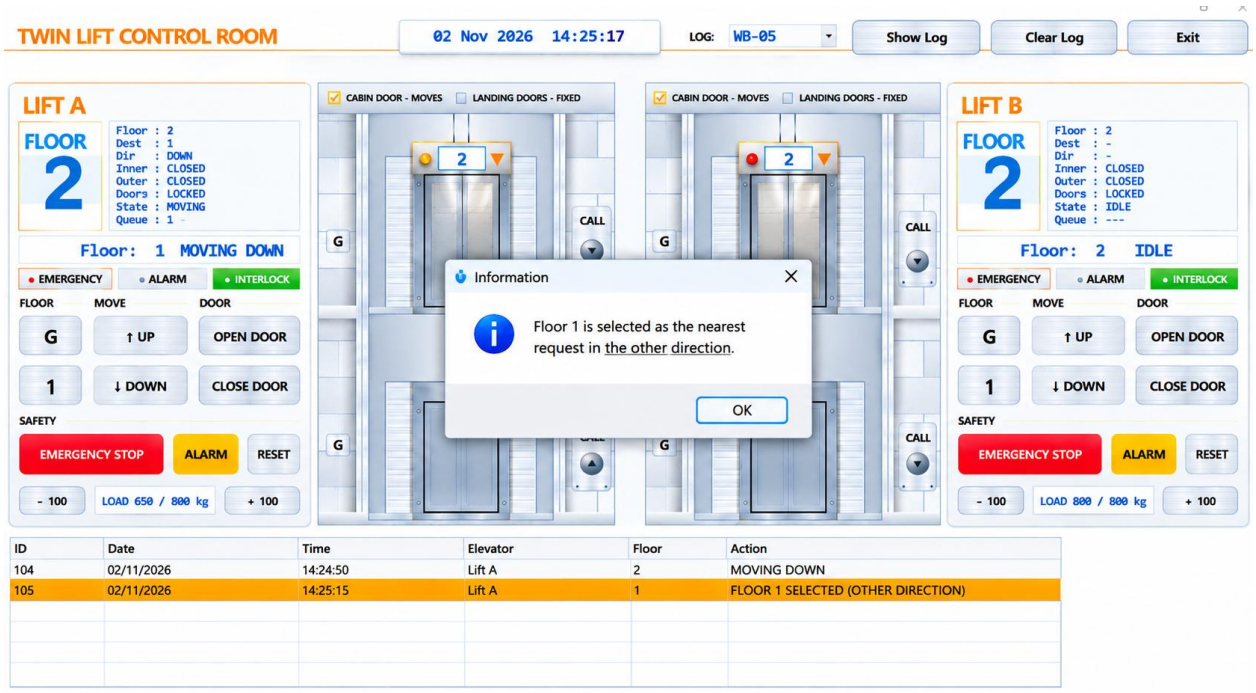


Fig:WB-05

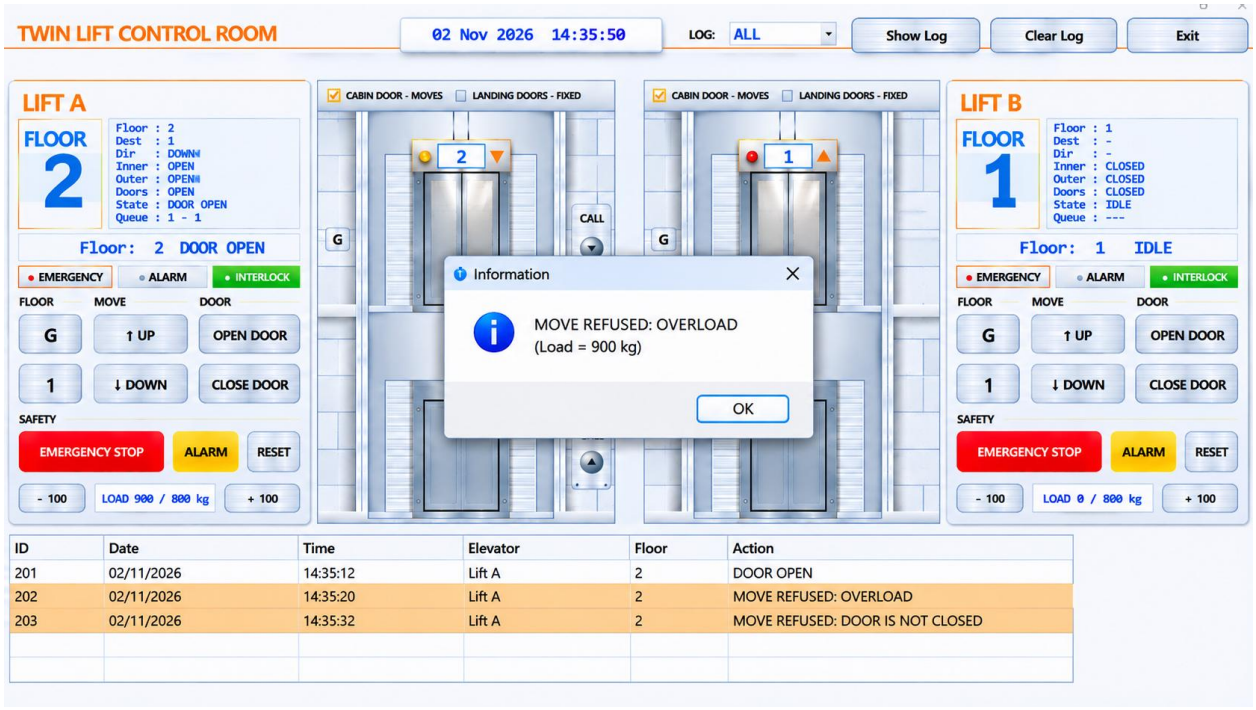


Fig: WB-06

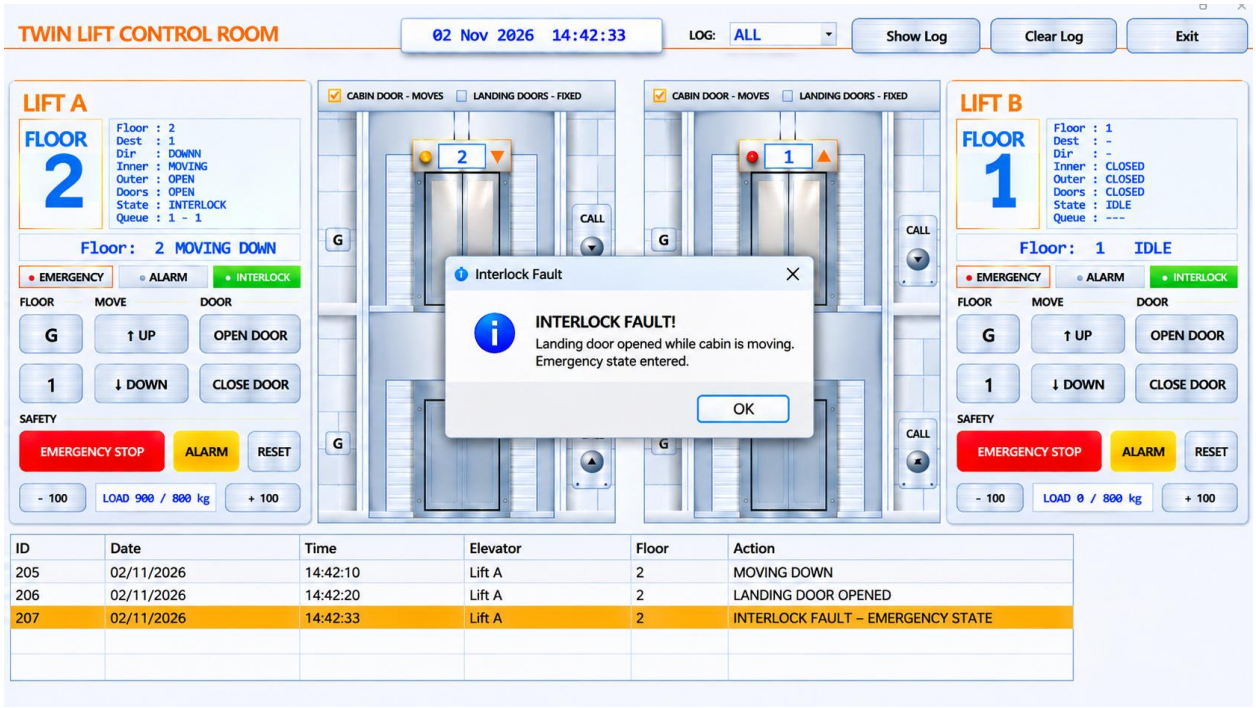


Fig:WB-07

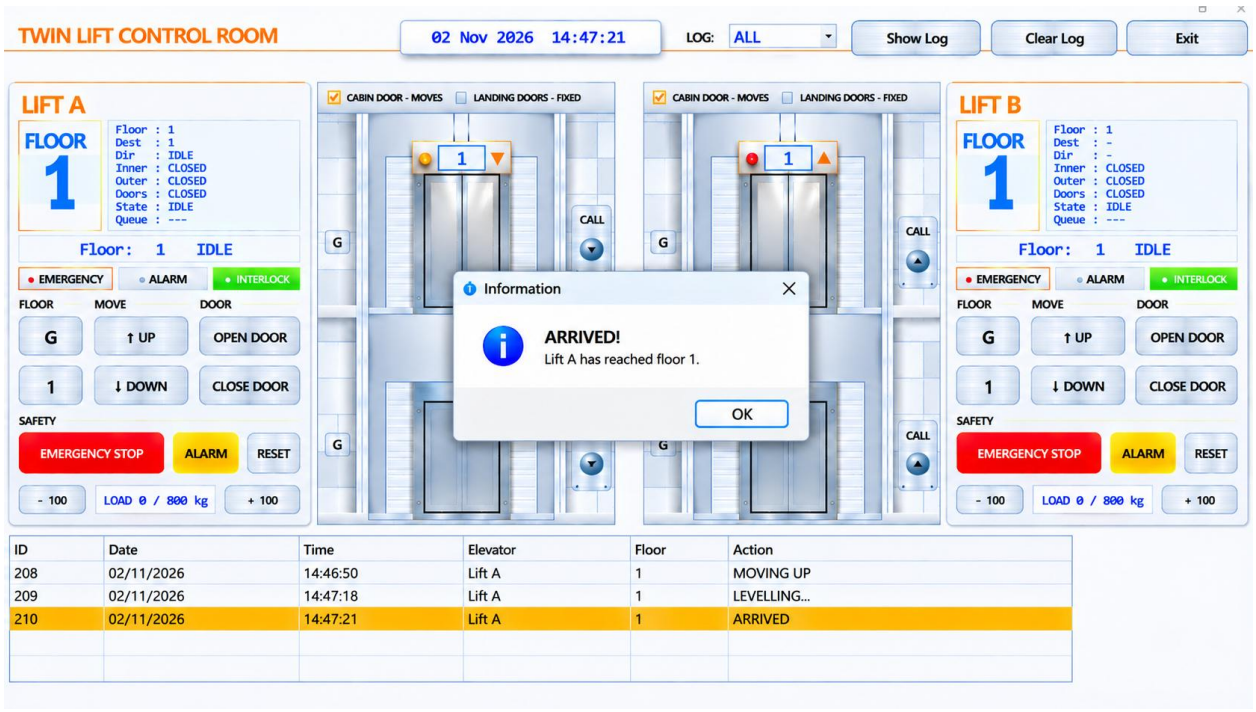


Fig: WB-08

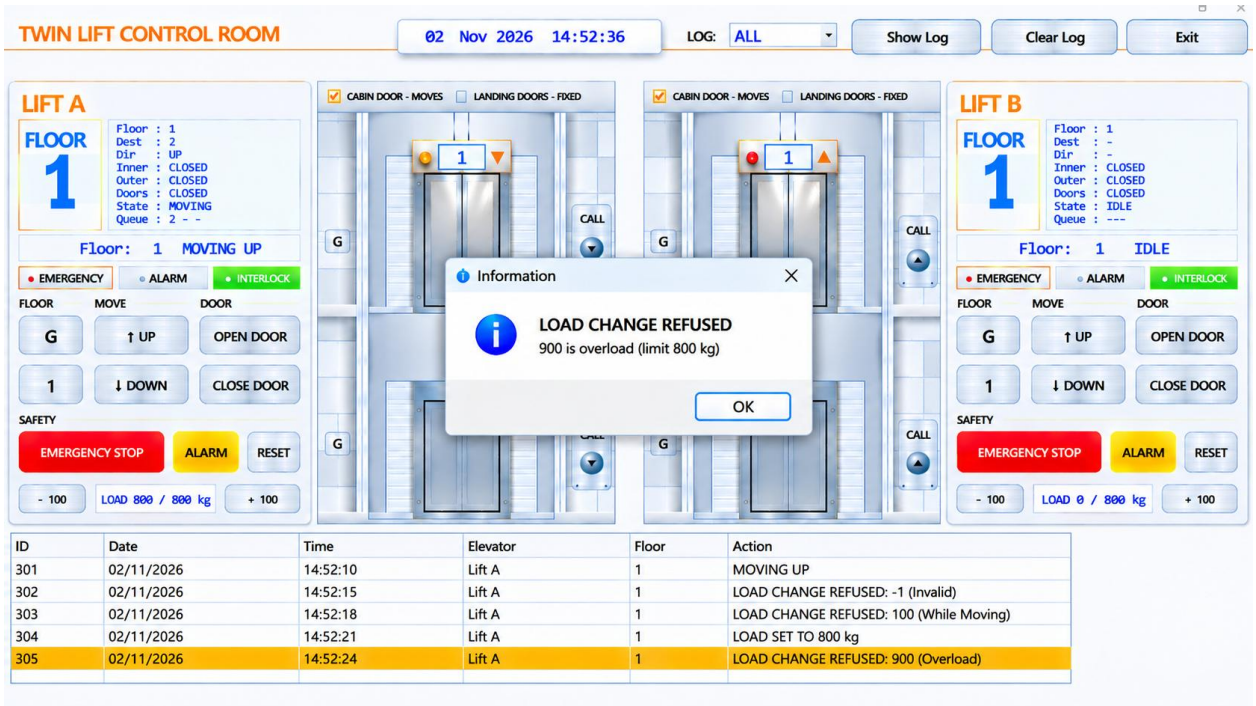


Fig:WB-09



Fig:WB-10

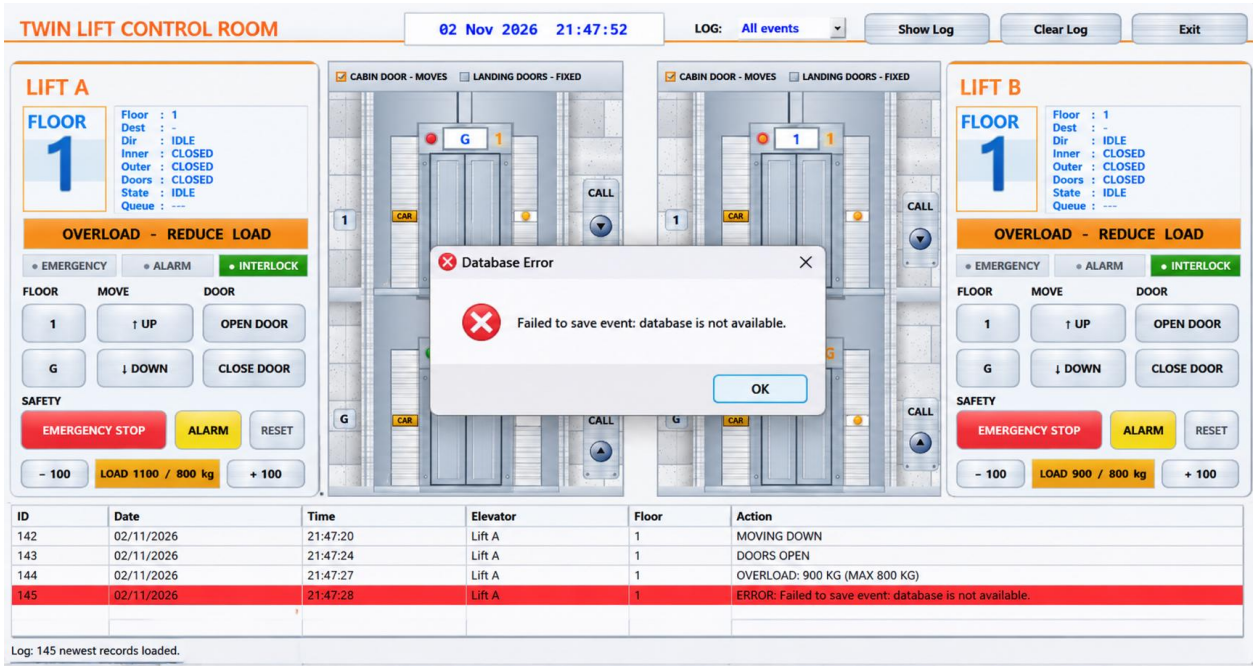


Fig:WB-11

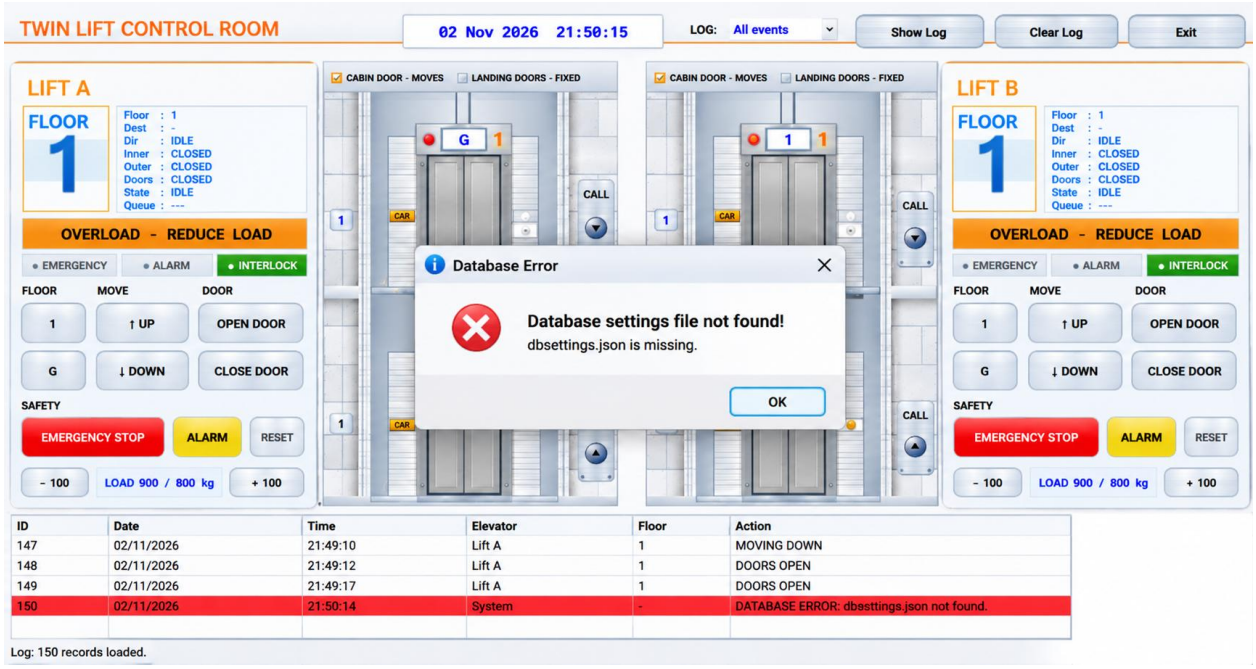


Fig:Wb-11

Coverage observation. With only two floors, the branch of RetargetWhileMoving that adds an intermediate stop cannot be reached, because the only other floor is the target. That branch can be tested only after MaxFloor is increased.

Summary

Out of the 27 test cases that are identified, 6 are covered through recordings, while 21 are still pending. The recording is only on the main behavior of the lift, and not the user interface nor the database, hence, these two are the weakest at the moment.

11. Critical Thinking

11.1 Advantages

- Safe from design: the interlock prevents motion if any door is not closed, and the state check occurs each tick, while emergency stop freezes the entire elevator.
- Fast response: animation works smoothly due to performing all database actions in separate threads.
- Isolated operation: each elevator has its unique model, controller, timer, alarm and panel to prevent propagation of malfunctioning across two elevators.
- Realistic doors: the interior doors move with the elevator and landing doors stay at their floors, but the current floor's landing door is opened only.
- Extendible: new elevator states, observers and floor numbers may be introduced without changing any class.
- Traceable: every action is recorded together with time, elevator and floor data and may be viewed using SQL queries.
- Convenient installation: the database and table are created during startup, and the settings are stored in one JSON file.
- Detailed user interface: the lamps, message string and inactive buttons allow for clear understanding of available options and reasons for their unavailability.

11.2 Limitations

- Only G and 1 floors are simulated. Floor range is constant but each lift has constant pairs of floor buttons, thus increasing number of floors requires change to the interface.
- No dispatcher for all elevators: each elevator operates separately so there is no possibility to assign calls to the nearest elevator.
- Simulation only: there are no sensors or any physical mechanisms, only the amount of load which can be changed by pressing some buttons.
- Credentials: there is the dbsettings.json file which contains a clear password for the MySQL database and is not ignored by .gitignore file. This file should not be submitted along with the code.
- Records shown are limited: grid shows only the newest 500 records filtered for lifts only.
- Limits: speeds, maximum load, and other timings are defined in the Elevator.cs file as constants and can not be changed during execution.
- Only Windows platform due to Windows Forms technology.
- No automated tests: all testing is done manually, according to the documentation, there were no builds in Visual Studio and no check on MySQL connection in the environment where simulation run.
- The constant window size (1480 x 790) may be inconvenient for the smaller screens.

11.3 Future Improvements

- Support a configurable number of floors and lifts from a configuration file and render the floor buttons dynamically from this file.
- Implement the group dispatcher to route each call to the optimal lift based on proximity and the least burden factor.
- Include unit tests for the state classes and dispatcher with mocks for the observer and a mock repository respectively.
- Obfuscate database credentials (user secrets or environment variables) and exclude the dbsettings.json file from source code control.
- Normalize the database; move lift name, floor names and event type information into lookup tables with severity values, as in the illustration below, albeit at the cost of additional joins in the log query.
- Implement reporting and charting of log data (e.g. trips by hour, floors with most activity, emergency events), pager alerts and CSV exporting facilities.
- Implement maintenance mode, fire service mode and priority calls.

```
CREATE TABLE elevator   (elevator_id TINYINT PRIMARY KEY, name VARCHAR(20) NOT NULL
UNIQUE);
CREATE TABLE floor      (floor_no TINYINT PRIMARY KEY, label VARCHAR(10) NOT NULL); CREATE TABLE
event_type (type_id SMALLINT PRIMARY KEY, description VARCHAR(80) NOT NULL,                severity
ENUM('Info','Warning','Critical') NOT NULL);
-- elevator_log would then reference these three tables with foreign keys (proposal only, not implemented)
```

12. Conclusion

The project produces the implemented control system for two individual lifts. Both cabin travels continuously according to acceleration, cruise, and levelling phases, internal and external doors are synchronised properly, interlock prevents any kind of unsafe movement (checked on each tick), and all the events are saved into the MySQL database. Application structure and design patterns including State and Observer, clear division of interface layer, control layer, and data access layer, and usage of BackgroundWorker maintain code architecture and interface responsiveness.
Application provides encapsulation, inheritance, polymorphism, abstraction, events handling and resources management. Key drawbacks of the application are low number of floors, no common dispatcher and lack of automated tests. Test documentation provided in Section 10 of the paper should be

completed before finalisation of the project. Nonetheless, current solution can be considered as a good starting point for the future development of the application with added functionality such as more number of floors, common dispatcher and automated tests.

References

- Ahn, S. L. S. a. B. H., 2017. A smart elevator scheduler that considers dynamic changes of energy cost and user traffic. *Integrated Computer-Aided Engineering*, 24(2), pp. 187-202.
- Ampatzoglou, A. K. A. K. G. a. S. I., 2011. An empirical investigation on the reusability of design patterns and software packages. *Journal of Systems and Software*, 84(12), p. 2265–2283.
- Code, P. S., 2026. README.md, CHANGES_Twin.md, WHITE_THEME_NOTES.txt, FIX_NOTES.txt and Database/elevator_db.sql. *Project Documentation / Source Code*, Volume N/a, p. N/a.
- Fowler, M., 2003. UML Distilled: A Brief Guide to the Standard Object Modeling Language. *Addison-Wesley*.
- Gamma, E. H. R. J. R. a. V. J., 1994. Design Patterns: Elements of Reusable Object-Oriented Software. *Addison-Wesley*, N/a(N/a), p. N/a.
- Microsoft, N.d. BackgroundWorker Class. *Microsoft Learn*, p. N/a. Sommerville, I., 2006. Software Engineering. *Pearson*, Volume N/a, p. N/a.

14. Marking Matrix with Self-Assessment

Table 16. Self-assessment against suggested criteria (suggested values)

Criterion	Max. marks	Obtained (suggested)	Comments
Introduction, problem and objectives	10	9	Problem, challenges, aim and measurable objectives are stated clearly.
Planning and requirements	10	8	Functional and non-functional requirements are traced to code; the weekly schedule is a suggested one and must be replaced with the real timeline.
System design and diagrams	15	13	Use case, class, ER, activity, state and architecture diagrams drawn from the real code.
Implementation (OOP, patterns, database)	25	22	State and Observer patterns, five OOP principles, parameterised SQL and background processing are implemented.
Prototype and user interface	10	8	Interface and behaviour are explained in full; screenshots must still be inserted.
Testing and evaluation	15	9	Test cases are complete, but only 6 of 27 have recorded results; no automated tests exist.
Critical thinking	5	4	Advantages, limitations (including a security weakness) and improvements are specific.
Documentation quality and referencing	10	8	Consistent structure, captions and genuine references.
Total	**100**	**81**	Suggested self-assessment; marks for testing and prototype can rise once results and screenshots are added.

15. Appendices

Appendix A – GitHub Link to Complete Source Code

Repository: https://github.com/hanselbogati7/Elivator-project.git

Important: before publishing, remove the database password from dbsettings.json or add dbsettings.json to .gitignore. The supplied .gitignore excludes only bin/, obj/, .vs/ and *.user

.Appendix B – Use Case Diagram

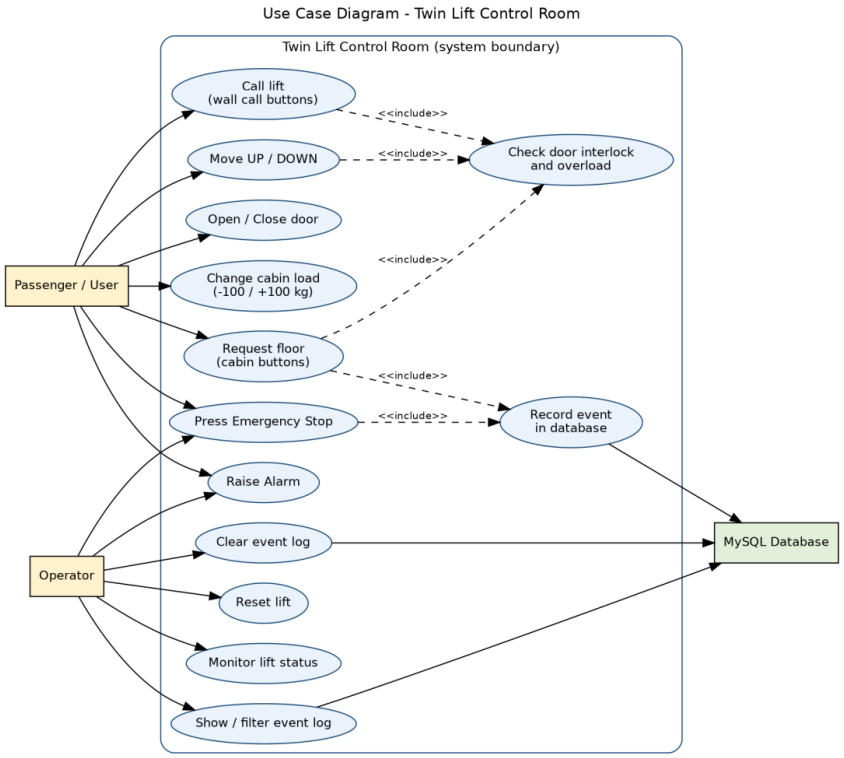
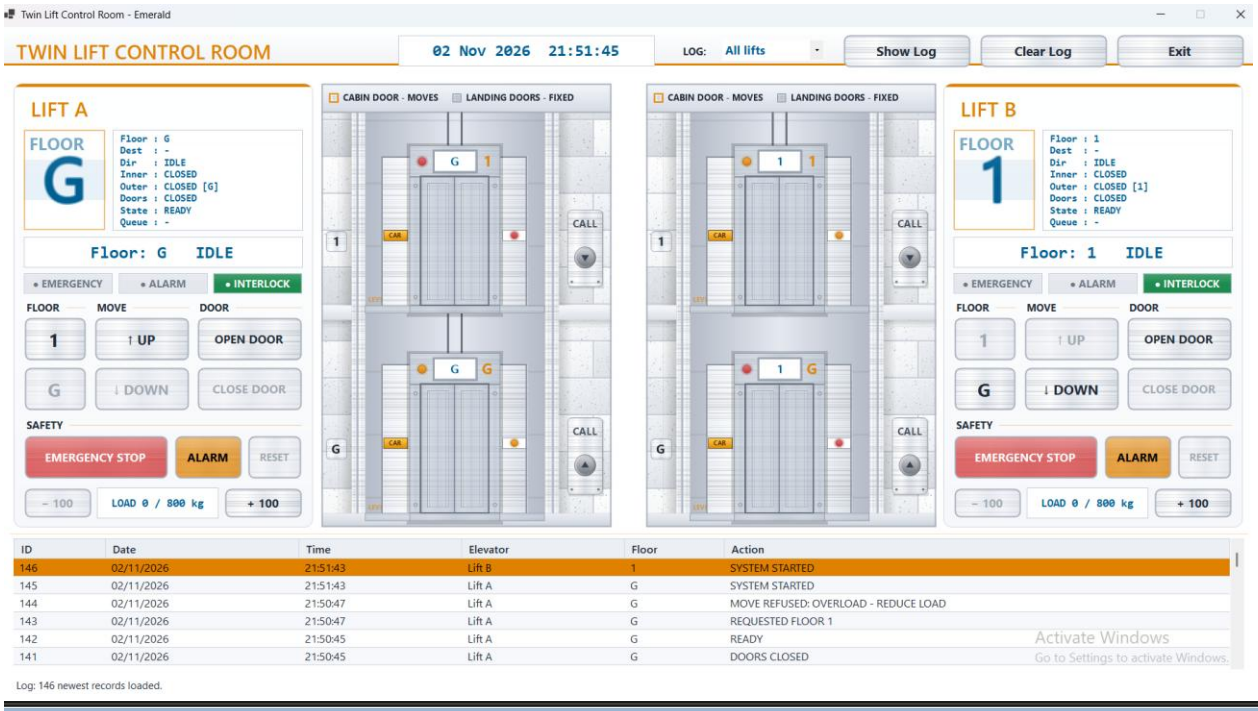


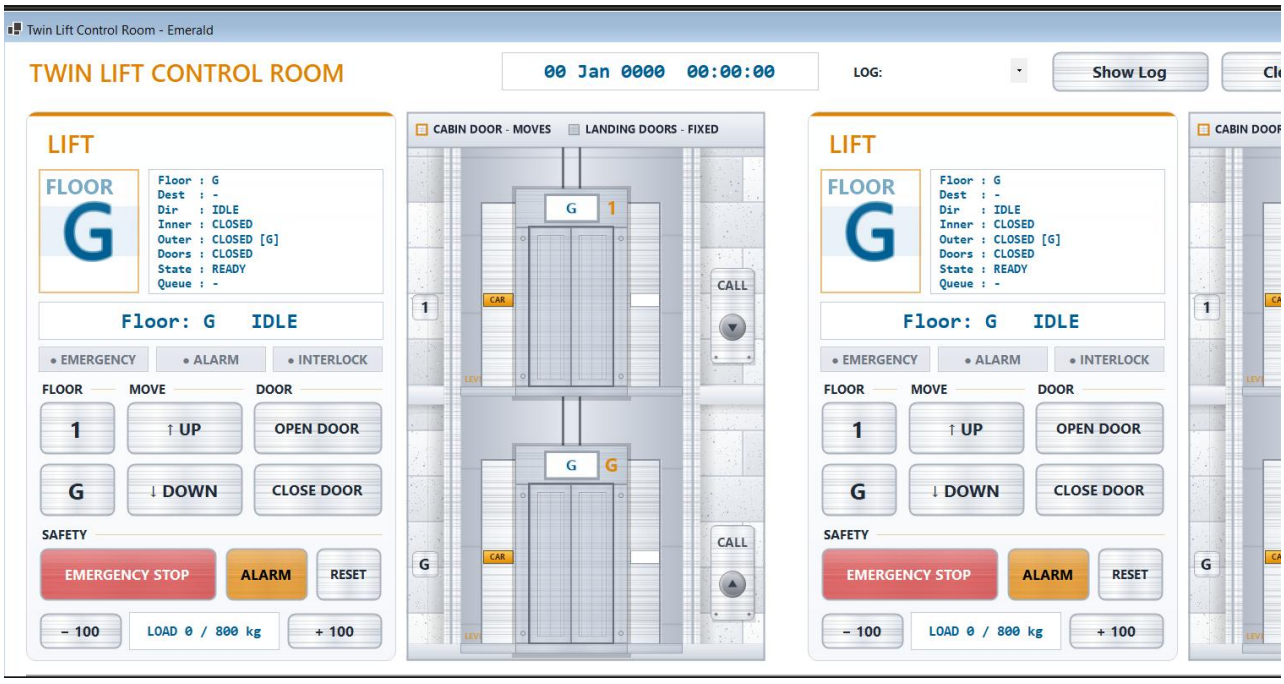
Figure 14. Use case diagram (repeated from Section 7.1)

Table 17. Use case descriptions

Use case	Actor	Description
Request floor	Passenger	Press a floor button in the cabin; the floor is queued unless an emergency stop is active.
Call lift	Passenger	Press a call button on the wall of the shaft; handled like a floor request.
Move UP / DOWN	Passenger	Request the next floor above or below the cabin.
Open / Close door	Passenger	Open or close the doors while the cabin stands at a floor.
Press Emergency Stop	Passenger, Operator	Freeze the lift, clear the queue and wait for RESET.
Raise Alarm	Passenger, Operator	Flash and sound the alarm for 6 s or until RESET.
Reset lift	Operator	Clear an emergency stop or alarm.
Change cabin load	Passenger	Simulate load in steps of 100 kg; above 800 kg is an overload.
Monitor lift status	Operator	Read the status list, lamps and message line.
Show / filter event log	Operator	Load the newest 500 records and filter them by lift.
Clear event log	Operator	Delete all records after confirmation.

Appendix C – Additional Screenshots


Twin Lift Control Room - Emerald
TWIN LIFT CONTROL ROOM
02 Nov 2026 21:51:45
LOG: All lifts
Show Log
Clear Log
Exit
LIFT A
FLOOR G
Floor : G
Dest : -
Dir : IDLE
Inner : CLOSED
Outer : CLOSED [G]
Doors : CLOSED
State : READY
Queue : -
Floor: G IDLE
• EMERGENCY • ALARM • INTERLOCK
FLOOR MOVE DOOR
1 ↑ UP OPEN DOOR
G ↓ DOWN CLOSE DOOR
SAFETY
EMERGENCY STOP ALARM RESET
– 100 LOAD 0 / 800 kg + 100
CABIN DOOR - MOVES LANDING DOORS - FIXED
CALL
LIFT B
FLOOR 1
Floor : 1
Dest : -
Dir : IDLE
Inner : CLOSED
Outer : CLOSED [1]
Doors : CLOSED
State : READY
Queue : -
Floor: 1 IDLE
ID Date Time Elevator Floor Action
146 02/11/2026 21:51:43 Lift B 1 SYSTEM STARTED
145 02/11/2026 21:51:43 Lift A G SYSTEM STARTED
144 02/11/2026 21:50:47 Lift A G MOVE REFUSED: OVERLOAD - REDUCE LOAD
143 02/11/2026 21:50:47 Lift A G REQUESTED FLOOR 1
142 02/11/2026 21:50:45 Lift A G READY
141 02/11/2026 21:50:45 Lift A G DOORS CLOSED
Activate Windows
Go to Settings to activate Windows.
Log: 146 newest records loaded.


Twin Lift Control Room - Emerald
TWIN LIFT CONTROL ROOM
00 Jan 0000 00:00:00
LOG:
Show Log
LIFT
FLOOR G
Floor : G
Dest : -
Dir : IDLE
Inner : CLOSED
Outer : CLOSED [G]
Doors : CLOSED
State : READY
Queue : -
Floor: G IDLE
• EMERGENCY • ALARM • INTERLOCK
FLOOR MOVE DOOR
1 ↑ UP OPEN DOOR
G ↓ DOWN CLOSE DOOR
SAFETY
EMERGENCY STOP ALARM RESET
– 100 LOAD 0 / 800 kg + 100
CABIN DOOR - MOVES LANDING DOORS - FIXED
CALL
LIFT
FLOOR G
Floor : G
Dest : -
Dir : IDLE
Inner : CLOSED
Outer : CLOSED [G]
Doors : CLOSED
State : READY
Queue : -
Floor: G IDLE
• EMERGENCY • ALARM • INTERLOCK
FLOOR MOVE DOOR
1 ↑ UP OPEN DOOR
G ↓ DOWN CLOSE DOOR
SAFETY
EMERGENCY STOP ALARM RESET
– 100 LOAD 0 / 800 kg + 100

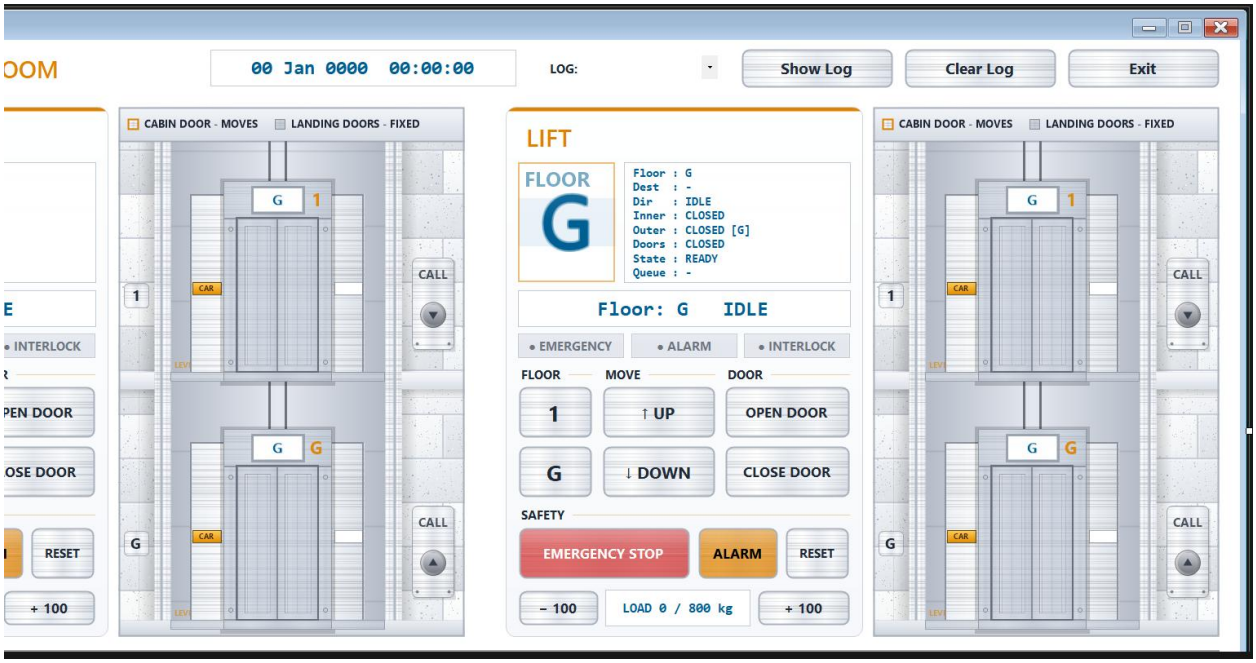


Figure 15. Additional screenshot (for example Lift B travelling down while Lift A is idle)

Appendix D – Additional Supporting Information

D.1 Project files

Table 18. Files of the Elevator_Twin project

File	Responsibility
Elevator.cs	Elevator, Door, Door State, interfaces, constants, request queue, door cycle, interlock, emergency, alarm
ElevatorStates.cs	IdleState, MovingState, EmergencyState
ElevatorController.cs	ElevatorController (UP/DOWN) and LogObserver
LiftUnit.cs, LiftUnit.Designer.cs	One complete lift: control card, shaft, its own Elevator, alarm timer and observers
Form1.cs, Form1.Designer.cs	Header, two LiftUnit controls, log grid, filter and clock
Controls.cs	Custom controls: ElevatorShaft, SteelButton, SteelPanel, CardPanel, LedDisplay, DisplayLabel, DisplayPanel, SectionLabel
Theme.cs	Colour theme (white theme; only this file needs to change to restyle)
LogRepository.cs	LogEntry, DbSettings, LogRepository: queue, BackgroundWorker, SQL, schema creation
Database/elevator_db.sql	Schema, view and example queries
dbsettings.json	Connection settings (host, port, user, password, database)
Elevator_Twin.csproj, .sln	Project definition (.NET 10, Windows Forms, MySqlConnector 2.4.0)

D.2 How to run

1. Start the MySQL server (127.0.0.1, port 3306) and check the settings in dbsettings.json.
2. Open Elevator_Twin.sln in Visual Studio. The first build needs internet access to restore the MySqlConnector package.

3. Press F5. The program creates the database and the table elevator_log on first use (running elevator_db.sql is optional).

D.3 Events written to the log

SYSTEM STARTED, REQUESTED FLOOR n, ALL DOORS LOCKED, GOING UP / GOING DOWN, APPROACHING, LEVELING, PASSING FLOOR n - LANDING DOOR STAYS LOCKED, STOP ADDED AT …,
ARRIVED AT …, DOORS UNLOCKED, INNER DOOR OPENING, … OUTER DOOR OPENING, DOORS OPEN,
DOORS CLOSING, DOORS CLOSED, READY, MOVE REFUSED …, REQUEST REFUSED …, OVERLOAD …, OVERLOAD CLEARED …, LOAD CHANGE REFUSED …, EMERGENCY STOP ACTIVATED, INTERLOCK FAULT …, ALARM ACTIVATED, ALARM STOPPED AUTOMATICALLY, RESET ….

D.4 Information still to be supplied by the student

- Cover-page details, declaration signature and acknowledgement names.
- The real weekly schedule (Section 5.1) and the hardware actually used (Section 6.1).
- Screenshots of the running application (Sections 9 and Appendix C).
- Actual results for all Pending test cases (Section 10).
- The GitHub repository link (Appendix A) and the official marking rubric (Section 14).